claude-windows / c47f0851-d75f-45b6-9126-e48544381426

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\c47f0851-d75f-45b6-9126-e48544381426\subagents\workflows\wf_5af9a132-4ce\agent-
a00d593a8e616af89.jsonl`
- SHA-256: `b6bdb154204c83a47f6af542f5ae1b4852247de24144423156271801d759e4cc`
- Source modified: `2026-06-20T09:03:01+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_5af9a132-4ce`
- session_id: `c47f0851-d75f-45b6-9126-e48544381426`
```

Transcript

1. user (2026-06-20T08:57:11.749Z)

```
Repo: badminton-highlight-indexer (Python). Branch feat/native-wasb-runner off origin/master.
The change builds the M3.5 Linux-native WASB detector runner + wires real extraction into the
worker.
Files changed/added (read them in the WORKING TREE ? changes are uncommitted):
- backend/pipeline/detectors/native_wasb_runner.py  (NEW ? the native runner)
- backend/pipeline/detectors/wasb_infer.py           (added --device, threaded device into
_build_cfg/run/run_video_streaming/main; device-keyed the resumable manifest)
- backend/pipeline/segmenters/trajectory_hybrid.py   (_resolve_runner: new 'native' branch +
_build_native_runner base hook; generalized 'WSL env' log lines)
- backend/pipeline/segmenters/wasb_hybrid.py         (_build_native_runner override)
- backend/pipeline/segmenters/fusion_hybrid.py       (_build_native_runner override; WASB
substrate only)
- backend/config/models.py                            (WasbIndexConfig.device + .python_bin;
detector_impl doc mentions 'native')
- backend/worker/__main__.py                          (cmd_train: --trajectory-source
{synth,detector} + --segmenter; _resolve_train_detector; detector path pulls videos + runs
runner)
- tests/test_native_wasb_runner.py, tests/test_worker.py (new tests)

DESIGN INTENT / CONTRACTS (judge against these):
- The EXISTING WSL runner is backend/pipeline/detectors/wasb_runner.py (WasbRunner,
WslCommandMixin). The native runner must be a sibling that runs the SAME wasb_infer.py natively
(no wsl, no 'conda activate' ? it invokes the wasb conda env python BY PATH, no /mnt path
translation) and emit a BYTE-IDENTICAL Frame,Visibility,X,Y CSV with the SAME filename
'{stem}__{vid12}_wasb.csv'.
- ACCEPTANCE GATE = byte-parity: same clip via WSL (Windows) vs native (Linux, device=cuda) ->
identical CSV + identical candidate windows. device default 'cuda' must keep wasb_infer's Hydra
overrides identical to before (parity).
- Selected via the detector_impl seam: indexing.detector_impl / RALLY_DETECTOR_IMPL = 'native'.
- Constraint: 'backend' must NOT import 'training' (worker shells out). The default trajectory
source stays 'synth' so the CPU e2e is unchanged.
- The code RUNS on Linux (GPU box) but the TESTS run on the dev's Windows machine; cross-
platform path handling matters.
- weights/repo/python/scratch come from env first
(WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH) then indexing.wasb.* then default.

YOU ARE THE SEAM/CONFIG SKEPTIC. Verify the detector_impl='native' wiring end-to-end:
_resolve_runner branch (env precedence, the alias set native/native_wasb/wasb_native), the base
_build_native_runner NotImplementedError, the wasb_hybrid + fusion_hybrid overrides, tracknet
falling through to the base refusal. Does WasbIndexConfig.device/python_bin actually survive
model_dump so NativeWasbConfig.from_indexing_cfg sees them (nested pydantic extra='ignore')?
Does env override beat config correctly (and does an empty env var fall through, not clobber)?
Any family where detector_impl='native' silently mis-resolves or crashes unexpectedly? Read
models.py + the segmenters.
```

2. assistant (2026-06-20T08:57:14.823Z)

I'll review the seam/config wiring for `detector_impl='native'`. Let me start by pulling the latest and reading the relevant files in the working tree.

3. user (2026-06-20T08:57:16.520Z)

```
feat/native-wasb-runner
 M backend/config/models.py
 M backend/pipeline/detectors/wasb_infer.py
 M backend/pipeline/segmenters/fusion_hybrid.py
 M backend/pipeline/segmenters/trajectory_hybrid.py
 M backend/pipeline/segmenters/wasb_hybrid.py
 M backend/worker/__main__.py
 M tests/test_worker.py
?? artifacts/
?? backend/pipeline/detectors/native_wasb_runner.py
?? docs/KHELSUTRA_PRODUCT_OVERVIEW.md
?? tests/test_native_wasb_runner.py
5c77f48 docs(decoupled): HANDOFF.md ? session checkpoint / resume-here (#255)
d5407eb feat(worker): Gen-0 train-from-bucket (worker train) on a CPU stub-trajectory mock (#254)
814d813 feat(deploy): GPU machine provisioning ? NVMe scratch + native WASB setup (#253)
```

4. assistant (2026-06-20T08:57:18.532Z)

Let me read the key files in the working tree.

5. user (2026-06-20T08:57:19.535Z)

```
"""Linux-native WASB shuttle-trajectory runner (no WSL).

The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
"Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md`` M3.5).
It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
Linux GPU box (GCP / a SaaS worker): it runs the **same** verified ``wasb_infer.py`` core
that the WSL adapter runs, so the trajectory CSV is byte-identical ? but WITHOUT the WSL
transport:

  * NO ``WslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
    ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
    (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
  * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps`` or ``cpu``; threaded
    into ``wasb_infer.py --device`` (and so into the Hydra ``runner.device`` override).
  * weights / repo / python / scratch come from the **environment** first
    (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
    ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.

It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
(``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``); ``detector_impl="native"``.

**Acceptance gate (owner / GPU box):** byte-parity ? the SAME clip through the WSL runner
(Windows) and this runner (Linux, ``device=cuda``) yields an identical trajectory CSV and
identical candidate windows. That parity is a hardware check (a real GPU + the ``wasb``
env), so it is verified by the owner on the box ? not in this offline, subprocess-mocked
test suite.
```

```
"""
import os
import shutil
import logging
import subprocess
from dataclasses import dataclass
from typing import Any, Dict, List, Optional, Tuple

from backend.pipeline.detectors.base import DetectorRunner
# Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to WSL/TrackNet.
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner

logger = logging.getLogger(__name__)

# Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
# detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
_INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


@dataclass
class NativeWasbConfig:
    """Resolved settings for a Linux-native WASB run.

    Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
    over ``indexing.wasb.*`` config (the box sets env; no config edits needed). ``device``
    defaults to ``cuda`` (the WSL byte-parity path).
    """
    repo_dir: str = "~/models/WASB-SBDT"   # native clone of nttcom/WASB-SBDT
    python_bin: str = "python"             # the `wasb` conda env python (invoked by path, no activate)
    weights_path: str = ""                 # REQUIRED (env WASB_WEIGHTS or indexing.wasb.weights_path)
    sport: str = "badminton"
    device: str = "cuda"                   # cuda | mps | cpu
    scratch_dir: str = ""                  # frame-extraction scratch (env); "" = next to the output dir
    timeout_sec: int = 1800               # 30 min; a hung GPU call is killed (0 = no timeout)
    keep_frames: bool = False              # retain the decoded frame cache after success (debug only)
    stream_video: bool = False             # fast path: decode in-process, no PNG extraction (bit-identical)

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
        w = (idx_cfg or {}).get("wasb", {}) or {}
        env = os.environ.get
        return cls(
            repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
            python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
            weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
            sport=w.get("sport", "badminton"),
            device=env("WASB_DEVICE") or w.get("device", "cuda"),
            scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or "",
            timeout_sec=int(w.get("timeout_sec", 1800)),
            keep_frames=bool(w.get("keep_frames", False)),
            stream_video=bool(w.get("stream_video", False)),
        )


class NativeWasbRunner(DetectorRunner):
    """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
```

```
docstring."""

    def __init__(self, cfg: NativeWasbConfig):
        self.cfg = cfg

    # --- low-level native exec (the single place tests patch) --------------------------
    def _run(self, argv: List[str], cwd: Optional[str] = None) -> subprocess.CompletedProcess:
        logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
        return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
                              timeout=(self.cfg.timeout_sec or None))

    def _repo_src(self) -> str:
        return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")

    def _copy_infer_script(self) -> bool:
        """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds configs/."""
        repo_src = self._repo_src()
        try:
            os.makedirs(repo_src, exist_ok=True)
            shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
            return True
        except OSError as e:
            logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
            return False

    def _rm_rf(self, path: Optional[str]) -> None:
        """Best-effort recursive delete of a native path (post-success cleanup; never raises)."""
        if path:
            shutil.rmtree(path, ignore_errors=True)

    def healthcheck(self) -> Tuple[bool, str]:
        """Verify weights + repo present and the `wasb` python imports torch (and sees a GPU on cuda)."""
        if not self.cfg.weights_path:
            return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or indexing.wasb.weights_path).")
        weights = os.path.expanduser(self.cfg.weights_path)
        if not os.path.exists(weights):
            return False, f"WASB weights not found at {weights}"
        repo_src = self._repo_src()
        if not os.path.isdir(repo_src):
            return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone nttcom/WASB-SBDT "
                           f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
        code = "import torch; print('torch', torch.__version__, 'cuda', torch.cuda.is_available())"
        try:
            res = self._run([self.cfg.python_bin, "-c", code])
        except FileNotFoundError:
            return False, f"python binary not found: {self.cfg.python_bin!r} (set WASB_PYTHON)."
        except subprocess.TimeoutExpired:
            return False, "native torch healthcheck timed out."
        out = (res.stdout or "").strip()
        if res.returncode != 0:
            return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or out).strip()}"
        if self.cfg.device == "cuda" and "cuda True" not in out:
            return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
        return True, out

```

```
    def run_predict(self, video_win_path: str, output_win_dir: str,
                    video_id: Optional[str] = None) -> Optional[str]:
        """Run WASB natively on a video. Returns the path to the trajectory CSV, or None.

        Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the WSL
        runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte the
        same downstream contract, so the parity comparison is apples-to-apples.
        """
        output_dir = os.path.abspath(output_win_dir)
        os.makedirs(output_dir, exist_ok=True)
        # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
        norm = str(video_win_path).replace("\\", "/")
        stem, _ext = os.path.splitext(os.path.basename(norm))
        # Content-derived key: same filename + different bytes -> different key (no cache reuse).
        key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
        out_csv = os.path.join(output_dir, f"{key}_wasb.csv")

        if not self._copy_infer_script():
            logger.error("Could not stage wasb_infer.py into the WASB repo src/.")
            return None

        weights = os.path.expanduser(self.cfg.weights_path)
        argv = [self.cfg.python_bin, "wasb_infer.py",
                "--video", os.path.abspath(str(video_win_path))]
        frames_dir: Optional[str] = None
        if self.cfg.stream_video:
            # Fast path: decode in-process, no PNG extraction. Output is bit-identical to disk.
            argv.append("--stream-video")
        else:
            scratch = self.cfg.scratch_dir or output_dir
            frames_dir = os.path.join(scratch, f"{key}_frames")
            argv += ["--frames_out_dir", frames_dir]
        argv += ["--weights", weights,
                 "--sport", self.cfg.sport,
                 "--device", self.cfg.device,
                 "--out", out_csv]

        logger.info("Running native WASB inference (device=%s) on %s ...",
                    self.cfg.device, os.path.basename(norm))
        try:
            res = self._run(argv, cwd=self._repo_src())
        except FileNotFoundError:
            logger.error("python binary not found: %r (set WASB_PYTHON).", self.cfg.python_bin)
            return None
        except subprocess.TimeoutExpired:
            logger.error("WASB native inference timed out after %ss.", self.cfg.timeout_sec)
            return None
        if res.returncode != 0:
            logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout or "")[-2000:])
            return None
        if not os.path.exists(out_csv):
            logger.error("WASB native finished but expected CSV not found at %s", out_csv)
            return None
        logger.info("WASB trajectory CSV: %s", out_csv)

        # Success ? the CSV is the durable output; the decoded frame cache is a pure
```

```
intermediate.
        # On any earlier failure we return above WITHOUT cleaning, so the cache survives a resume.
        # (Streaming never materializes frames, so there is nothing to drop on that path.)
        if not self.cfg.keep_frames and frames_dir:
            logger.info("Cache hygiene: removing frame cache for %s "
                        "(set indexing.wasb.keep_frames=true to retain).", key)
            self._rm_rf(frames_dir)
        return out_csv

    # Trajectory parsing + windowing are inherited from tracknet_runner (model-agnostic).
    parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
    trajectory_to_action_windows = staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

6. user (2026-06-20T08:57:20.001Z)

```
"""Pydantic models for the application configuration.

This module defines the canonical, validated, typed configuration for the
Badminton Highlight Indexer. Built-in defaults live here (single source of truth).

The models support graceful migration from old config.json files via extra="allow".
"""

from __future__ import annotations

from typing import Any, List, Literal, Optional

from pydantic import BaseModel, Field, field_validator, model_validator

Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]


class ValidationRelevanceConfig(BaseModel):
    court_green_lower: list[int] = Field(default=[35, 40, 40])
    court_green_upper: list[int] = Field(default=[85, 255, 255])
    court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)


class ValidationConfig(BaseModel):
    min_resolution_width: int = 1280
    min_resolution_height: int = 720
    min_fps: float = 29.9
    min_blur_threshold: float = 20.0
    max_scene_cuts_per_minute: float = 0.5
    relevance: ValidationRelevanceConfig = Field(default_factory=ValidationRelevanceConfig)


class TrackNetConfig(BaseModel):
    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/TrackNetV4"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "TrackNetV4"
    weights_path: str = ""
    queue_length: int = 5
    stage_in_wsl: bool = True
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # Cache hygiene: after a SUCCESSFUL run the staged video copy (and, for WASB, the
```

```
    # decoded frame cache) are deleted automatically ? they are pure intermediates,
    # regenerable from the source, and the dominant disk consumer (~GBs/video). Set
    # true only for debugging. On FAILURE they are always kept so a re-run can resume.
    keep_frames: bool = False
    # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB.
    # 0 = disabled.
    min_free_gb: float = 0.0


class WasbIndexConfig(BaseModel):
    """Typed config for the live WASB shuttle substrate (indexing.wasb).

    Mirrors backend/pipeline/detectors/wasb_runner.py::WasbConfig.from_indexing_cfg so
    these knobs actually take effect ? previously this whole block was silently dropped
    because nested config sections defaulted to extra='ignore'.
    """
    wsl_distro: str = "Ubuntu"
    repo_dir: str = "~/models/WASB-SBDT"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "wasb"
    weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
    sport: str = "badminton"
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; kills a hung WSL/GPU call (0 = no timeout)
    velocity_threshold: float = 0.02
    # --- Linux-native runner only (detector_impl='native'); ignored by the WSL runner. ---
    # Env vars override these (WASB_DEVICE / WASB_PYTHON) for a config-free GPU box.
    device: str = "cuda"        # torch device for the native runner: cuda | mps | cpu
    python_bin: str = "python"  # the `wasb` conda env python (invoked by path, no `conda activate`)
    # Cache hygiene: after a SUCCESSFUL run the staged video copy + decoded frame cache
    # (the ~GBs/video disk hog) are deleted automatically ? pure intermediates, regenerable
    # from the source. Set true only for debugging. On FAILURE they are kept so a re-run resumes.
    keep_frames: bool = False
    # Warn before a run if WSL free space (at wsl_stage_dir) is below this many GB. 0 = disabled.
    min_free_gb: float = 0.0
    # FAST PATH (default OFF until owner-verified side-by-side): decode the staged video
    # on the fly and feed frames straight to the detector, skipping the slow per-frame PNG
    # extraction that dominates a long clip (~46k PNGs for a 12-min 1080p60 video, which
    # overran the 30-min timeout). Output is bit-identical to the PNG path (PNG is lossless),
    # so this is purely a speed/disk win; kept opt-in until the side-by-side confirms it.
    stream_video: bool = False


class FusionConfig(BaseModel):
    """Config for the multi-signal `fusion_hybrid` segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

    Every default reproduces control behaviour: the fusion segmenter persists the
    shuttle net-crossing count (Gate A signal) and ? only when ``cue_flags['person']`` is
    true ? the person-cue features, but it does NOT gate the served handoff unless a
    threshold is raised (``min_crossings``/``min_players`` default 0 = OFF). The court mask
    is optional: ``court_polygon=None`` ? Gate B counts every detection (player-count-only);
    supplied ? off-court persons (spectators / adjacent courts) are excluded.
```

```
    """
    # Which trajectory substrate seeds the windows (shuttle stays primary).
    substrate: Literal["wasb", "tracknet"] = "wasb"
    # Windowing velocity threshold for the fusion family (the engine reads
    # indexing.<family>.velocity_threshold). Default matches the substrates' 0.02; set
    # equal to indexing.wasb.velocity_threshold to A/B fusion against a tuned wasb run.
    velocity_threshold: float = 0.02
    # Per-cue enable flags, e.g. {"person": true}. Person cue is OFF unless explicitly
    # enabled ? so the default path never imports torch / touches the GPU.
    cue_flags: dict[str, bool] = Field(default_factory=dict)
    # Served Gate A: drop windows whose shuttle net-crossings < this from the AI handoff.
    # 0 = OFF (no gating; persisted candidates are always the full superset for offline re-scoring).
    min_crossings: int = Field(default=0, ge=0)
    # Served Gate B: when the person cue is on, drop windows with median in-court players
    # < this. 0 = OFF.
    min_players: int = Field(default=0, ge=0)
    # Optional court mask as normalised 0-1 [[x, y], ...] polygon. None = no mask.
    court_polygon: Optional[list[list[float]]] = None
    # Net line for the person two-sided-motion split (person features only ? the shuttle
    # net-crossing gate keeps rally_gate's camera-agnostic auto-estimate).
    net_axis: Literal["x", "y"] = "y"
    net_position: float = Field(default=0.5, ge=0.0, le=1.0)


class IndexingConfig(BaseModel):
    default_segmenter: str = "yolo_hybrid"
    use_gpu: bool = True
    # Detector backend for the trajectory hybrids (wasb/tracknet). "auto" = the real
    # WSL runner (default, production); "stub" = a deterministic CPU mock (no GPU/WSL)
    # so the whole pipeline is regression-testable on a CPU box / CI; "native" = the
    # Linux-native WASB runner (no WSL/conda-activate; a GPU box ? M3.5). The
    # RALLY_DETECTOR_IMPL env var overrides this. See pipeline/detectors/{stub,native_wasb}_runner.py.
    detector_impl: str = "auto"
    motion_threshold: float = 0.08
    min_segment_duration: float = 3.0
    dead_time_merge_gap: float = 2.0
    chunk_duration: float = 90.0
    chunk_overlap: float = 10.0
    detector_model: str = "fasterrcnn_resnet50_fpn"
    detector_score_threshold: float = 0.5
    yolo_velocity_threshold: float = 0.15
    yolo_frame_skip: int = 3
    yolo_tracker: str = "bytetrack.yaml"
    yolo_prefetch_workers: int = 2
    min_action_window_duration: float = 2.0
    # BOUNDED WINDOWING (over-merge fix W1, RALLY_QUALITY_RESEARCH.md §6). 0 = OFF; > 0 = a window
    # longer than this many seconds is recursively split at its largest internal inactivity gap (?
    # ``window_min_split_gap`` s ? the most likely rally boundary), so one window can't swallow
    # several rallies + the dead time between them. Never merges, only cuts (recall can't drop).
    # *** R1 MILESTONE DEFAULT-ON (cap=6): the smallest safe local-windowing flip. Measured n=13
    # golden, R2 tolF1: baseline?milestone (cap=6 + R4 below) ?+0.222, sign 12/0, p=0.0005; cap=6
    # beats cap=12 ?+0.110, sign 12/0; net over-seg guard PASS (mean merge_rate 0.651?0.161). ***
    max_window_duration: float = 6.0
    window_min_split_gap: float = 0.5  # min internal gap (s) that qualifies as a split
```

```
point
        # HARD-CAP fallback for W1: when True, an over-long window with NO internal inactivity gap
        # (a continuously-active trajectory ? e.g. the real-footage mahadevpura-1 174s blob) is
        # chopped at its midpoint until ? max_window_duration, making the cap a true upper bound.
        # Only cuts (recall can't drop). OFF by default until measured/promoted.
        window_hard_cap: bool = False
        # R4 rally-END inactivity trim (s, 0 = OFF). After a rally the shuttle keeps moving slowly
        # (picked up / walked) above velocity_thresh, so the window over-extends its END (the measured
        # |?end| gap vs golden). Trim the post-rally tail back to the last frame whose trailing
        # window stays dense with FAST motion (velocity > inactivity_rally_thresh). Only cuts.
        # *** R1 MILESTONE DEFAULT-ON (1.5/0.20/0.30): on top of cap=6, R4 adds a small but significant
        # end-precision gain ? ?tolF1 +0.040, sign 9/1/3, p=0.022, |?end| 4.96?3.31s (n=13). The W1 cap
        # is the dominant lever; R4 is the marginal increment. See QUALITY_ITERATIONS.md "R1". ***
        window_inactivity_end: float = 1.5
        inactivity_rally_thresh: float = 0.20
        inactivity_min_density: float = 0.30
        # R5 blob-fallback (default OFF). LAST-RESORT splitter for the continuously-active blob: after
        # the W1 gap-split AND the R4 inactivity trim have each had their chance, a group STILL longer
        # than window_blob_factor × max_window_duration (no internal gap, no rally-end signal ? e.g.
        # mahadevpura-1's ~65s residual ? 11× a 6s cap) is midpoint-chopped to ? the cap as a FORCED cut
        # (logged + flagged on the window) so the degenerate single-window outcome is avoided and the
        # clip is surfaced as needing rally-STATE cues, not a bigger cap. Differs from window_hard_cap
        # (which chops BEFORE R4 and over-segments normal clips): blob-fallback runs AFTER R4 and only
        # force-cuts the genuine blob (the factor gate). Only cuts (recall can't drop).
        window_blob_fallback: bool = False
        # Magnitude gate that keeps R5 surgical: only a group longer than this × max_window_duration is a
        # blob (a normal long rally ~1-2× the cap is left alone; the mahadevpura-1 residual is ~11×).
        # Default 4.0 is do-no-harm on the golden corpus (rescues only the continuously-active
        # clips, every other clip a no-op within noise) ? see docs/QUALITY_ITERATIONS.md "R5".
        window_blob_factor: float = 4.0
        log_yolo_candidates: bool = True
        store_yolo_candidates: bool = True
        ai_handoff_padding: float = 2.0
        ai_max_workers: int = 5
        # Width (px) the gemini segmenter re-encodes chunks to before upload. Stream-copied
        # chunks of a high-bitrate (4K) source blow past the provider upload cap
        # (backend/providers/gemini.py::MAX_UPLOAD_MB) and are refused ? observed live as
        # "0 segments / success". Re-encoding also makes chunk boundaries frame-accurate
        # (stream copy cuts at the nearest keyframe). 0 = legacy stream-copy at source res.
        # Matches gemini_refine's --clip-scale-width default.
        ai_chunk_scale_width: int = 1280
        # FULLY-LOCAL mode (default OFF): when true, the trajectory-hybrid segmenters
        # (wasb_hybrid / tracknet_hybrid / fusion_hybrid) skip the Phase-3 AI handoff entirely and
```

```
    # emit their local candidate windows AS the rallies ? no provider, no API key, nothing
    # uploaded. Lets the local detector run standalone (e.g. to benchmark it against the Gemini
    # path, or to operate with zero cloud dependency). The pure `gemini` segmenter ignores this.
    skip_ai_handoff: bool = False
    # Optional debug knob: trim every video to the first N seconds before processing.
    debug_duration: Optional[float] = None

    tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
    wasb: WasbIndexConfig = Field(default_factory=WasbIndexConfig)
    fusion: FusionConfig = Field(default_factory=FusionConfig)

    @field_validator("default_segmenter")
    @classmethod
    def segmenter_must_be_registered(cls, v: str) -> str:
        # Registry check happens at runtime in main/segmenter selection.
        # Here we just allow any string for forward compatibility.
        return v

    @model_validator(mode="after")
    def _chunk_step_must_be_positive(self) -> "IndexingConfig":
        # The chunked segmenters advance by (chunk_duration - chunk_overlap); a non-positive
        # step makes `while t < duration: t += step` loop forever, growing memory before any
        # GPU work. Reject the bad config at load time instead.
        if self.chunk_overlap >= self.chunk_duration:
            raise ValueError(
                f"indexing.chunk_overlap ({self.chunk_overlap}) must be < chunk_duration "
                f"({self.chunk_duration}); otherwise chunking never advances."
            )
        return self


class OutputConfig(BaseModel):
    default_highlights_name: str = "highlights.mp4"
    db_name: str = "sports_indexer.db"
    # Directory where the DB, highlight reels, and processed clips are written.
    # The CLI's --output-dir overrides this at startup (see backend/main.py::main).
    output_dir: str = "output"


class WatermarkConfig(BaseModel):
    """Branding overlay burned into each highlight clip during the per-clip re-encode
    (backend/pipeline/stitcher/compiler.py). **On by default** ? set `enabled: false`
    (under `stitching.watermark` in config.json) to turn it off. The text is fully
    configurable. The concat stage stays stream-copy (-c copy)."""
    enabled: bool = True
    text: str = "Generated by Khelsutra.guru"
    logo_path: str = ""        # optional transparent PNG overlay; "" = no logo
    font_path: str = ""        # optional .ttf; "" = use the fontconfig `font` family below
    font: str = "Sans"         # fontconfig family used when font_path is empty
    font_size_ratio: float = Field(default=0.035, gt=0.0, le=0.5)  # text height as a fraction of frame height
    opacity: float = Field(default=0.85, ge=0.0, le=1.0)
    box: bool = True           # faint backing box behind the text for legibility
    margin: int = Field(default=24, ge=0)  # px inset from the chosen corner
    position: Literal["bottom-right", "bottom-left", "top-right", "top-left"] = "bottom-right"


```

```
class StitchingConfig(BaseModel):
    begin_padding: float = 0.5
    end_padding: float = 0.5
    use_cache: bool = False
    min_shot_count: int = 1
    # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose duration
    # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
    # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on the
    # fully-local no-AI path, whereas duration is derived from the rally's own start/end. The
    # local detector over-fires and its false positives are mostly SHORT (motion blips,
    # background-court fragments), so this keeps the reel to the eye-catching long rallies.
    # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
    min_rally_duration: float = Field(default=0.0, ge=0.0)
    watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


class LoggingConfig(BaseModel):
    """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
    # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
    # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
    # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
    # them to WARNING; set true to restore full chatter for request-flow debugging.
    verbose_http: bool = False


class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]


class StorageConfig(BaseModel):
    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
    behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO Spaces
    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = Google Drive
    (follow-up). Per-backend settings are loose dicts passed to the backend constructor ?
    **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
    # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
    output_root: str = ""
    local: dict[str, Any] = Field(default_factory=dict)
    s3: dict[str, Any] = Field(default_factory=dict)
    gcs: dict[str, Any] = Field(default_factory=dict)
```

```
        gdrive: dict[str, Any] = Field(default_factory=dict)


    class AppConfig(BaseModel):
        """Root configuration object.

        All runtime configuration should be accessed via an instance of this class
        (or the loader) rather than raw dict lookups.
        """

        sport: Sport = "badminton"
        ai_provider: str = "gemini"
        # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable flagship;
        # the alias never deprecates underneath us. Eval reports record the run date alongside.
        ai_model: str = "gemini-flash-latest"

        validation: ValidationConfig = Field(default_factory=ValidationConfig)
        indexing: IndexingConfig = Field(default_factory=IndexingConfig)
        output: OutputConfig = Field(default_factory=OutputConfig)
        stitching: StitchingConfig = Field(default_factory=StitchingConfig)
        security: SecurityConfig = Field(default_factory=SecurityConfig)
        logging: LoggingConfig = Field(default_factory=LoggingConfig)
        storage: StorageConfig = Field(default_factory=StorageConfig)

        # Allow extra keys during transition so existing config.json files do not break.
        model_config = {
            "extra": "allow",
            "validate_assignment": True,
        }

        # Temporary migration shim for legacy dict-style access (config.get("section", {}).get(...)).
        # Will be removed once all call sites (validators, segmenters, etc.) move to typed
        # attribute access / the typed config. A nested *section* is returned as a plain dict
        # via model_dump, so legacy chained access keeps working ? returning the Pydantic
        # section model instead breaks it (a model has no .get), which bit every validator on
        # real runs. Scalar leaves are returned as-is.
        def get(self, key: str, default: Any = None) -> Any:
            _MISSING = object()
            val: Any = _MISSING
            if key in type(self).model_fields or hasattr(self, key):
                val = getattr(self, key, _MISSING)
            if val is _MISSING:
                # Legacy convenience: find a leaf key inside any nested section. Sections are
                # discovered dynamically from the model fields, so a future top-level section
                # (e.g. `audio`, `jobs`) is covered automatically without editing a hardcoded list.
                for fname in type(self).model_fields:
                    section = getattr(self, fname, None)
                    if isinstance(section, BaseModel) and hasattr(section, key):
                        val = getattr(section, key)
                        break
            if val is _MISSING:
                return default
            if isinstance(val, BaseModel):
                return val.model_dump(mode="json")
            return val

        def __getitem__(self, key: str) -> Any:
```

```
        val = self.get(key)
        if val is None and key not in type(self).model_fields:
            # Re-raise like a real dict for unknown top-level
            raise KeyError(key)
        return val


# Convenience type alias for callers that want to annotate
Config = AppConfig

```

7. assistant (2026-06-20T08:57:23.021Z)

Now let me read the segmenters to verify the `_resolve_runner` branch and `_build_native_runner` overrides.

8. user (2026-06-20T08:57:23.641Z)

```
"""Shared engine for trajectory-substrate hybrid segmenters.

tracknet_hybrid and wasb_hybrid were copy-paste twins ? CODE_MAP carried a
standing footgun: "any P3/P4 fix must be applied to BOTH". This base class is
now the single home for the whole pipeline shape:

    healthcheck gate -> P2 trajectory -> candidate windows (+ persistence)
    -> P3 AI-provider handoff over padded clips -> P4 DB population/linking

A concrete subclass supplies only its identity and runner wiring:
- ``family``        ? config section under ``indexing.*``, output subdir, and
                      the ``<family>-hybrid-<model>`` detector tag in metadata.
- ``display_label`` ? human label for log lines ("WASB", "TrackNet").
- ``name`` / ``description`` properties (the registry contract).
- ``_build_runner(idx_cfg)`` ? returns (DetectorRunner, detector-specific
                      detection_params extras such as weights_path).

This base is deliberately NOT registered; only concrete subclasses register.
"""
import os
import time
import logging
import concurrent.futures
from typing import Any, Dict, List, Optional, Tuple

import cv2

from backend.pipeline.segmenters.base import BasePlaySegmenter
from backend.utils.video import get_video_duration, extract_video_chunk
from backend.sports import sport_registry
from backend.providers import provider_registry
from backend.pipeline.detectors.base import DetectorRunner

logger = logging.getLogger(__name__)


def _fmt_ts(seconds: float) -> str:
    seconds = max(0, int(seconds))
    h, rem = divmod(seconds, 3600)
    m, s = divmod(rem, 60)
    return f"{h:02d}:{m:02d}:{s:02d}"


class TrajectoryHybridSegmenter(BasePlaySegmenter):
    """Trajectory-detector hybrid: precise candidate rallies + AI semantic boundaries."""

```

```python
    #: config section under `indexing` (velocity_threshold lives there), the
    #: output subdir for trajectories, and the metadata detector-tag prefix.
    family: str = ""
    #: human-readable label used in log lines.
    display_label: str = ""

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, detector-specific detection_params extras) for the default (WSL) path."""
        raise NotImplementedError

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Return (runner, extras) for the Linux-native (no-WSL) path. Only families with a
        native runner override this; the base refuses with a clear message (M3.5: WASB only)."""
        raise NotImplementedError(
            f"detector_impl='native' is not supported for the "
            f"{self.display_label or self.family or 'trajectory'!r} family ? only WASB has a "
            f"Linux-native runner. Use detector_impl='auto' (WSL) or 'stub'.")

    def _resolve_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        """Resolve the detector runner, honouring the CPU-stub and Linux-native overrides.

        ``RALLY_DETECTOR_IMPL`` env var (takes precedence) or ``indexing.detector_impl``:
        ``"stub"`` swaps in a deterministic CPU runner (no GPU/WSL) so the whole pipeline
        is regression-testable without a GPU ? "mock a GPU with a CPU"; ``"native"`` swaps
        in the Linux-native WASB runner (no WSL/conda-activate; GPU box) via
        ``_build_native_runner``. Anything else (``"auto"``) delegates to ``_build_runner``
        (the WSL runner) so the default production path is unchanged
        (``docs/DECOUPLED_COMPUTE/``).
        """
        impl = (os.environ.get("RALLY_DETECTOR_IMPL") or idx_cfg.get("detector_impl") or "auto").lower()
        if impl == "stub":
            from backend.pipeline.detectors.stub_runner import StubDetectorRunner, StubConfig
            logger.info("%s: detector_impl=stub ? CPU mock runner (no GPU/WSL).",
                        self.display_label or "trajectory")
            return StubDetectorRunner(StubConfig.from_indexing_cfg(idx_cfg)), {"detector_impl": "stub"}
        if impl in ("native", "native_wasb", "wasb_native"):
            logger.info("%s: detector_impl=native ? Linux-native runner (no WSL).",
                        self.display_label or "trajectory")
            return self._build_native_runner(idx_cfg)
        return self._build_runner(idx_cfg)

    @staticmethod
    def _probe_fps_width(video_path: str) -> tuple:
        """Return (fps, frame_width) for a video, with sensible fallbacks."""
        cap = cv2.VideoCapture(video_path)
        fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
        width = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
        cap.release()
        return (fps if fps > 0 else 30.0, width if width > 0 else 1280.0)

```

```
    @staticmethod
    def _shot_count_from(segment: Dict[str, Any], duration: float) -> int:
        """Provider-reported exchange count, else a duration-based estimate.

        One canonical implementation ? the twins had drifted (wasb relied on
        ``int(None)`` raising into the fallback; same result, by accident).
        """
        shot_count = segment.get("shuttle_exchange_count")
        if shot_count is None:
            return max(3, int(duration / 0.8))
        try:
            return int(shot_count)
        except (ValueError, TypeError):
            return max(3, int(duration / 0.8))

    # --- Fusion seams (identity by default ? wasb_hybrid/tracknet_hybrid unchanged) -------
    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        """Stats dict persisted into ``candidate_segments.stats`` for one window. The BASE
        returns exactly the 4 motion keys, so the trajectory twins persist byte-identical
        rows; ``FusionSegmenter`` overrides this to add ``net_crossings`` (+ person-cue
        features). ``points`` are the whole-video trajectory points (TrajectoryPoint)."""
        return {
            "peak_velocity": cw["peak_velocity"],
            "mean_velocity": cw["mean_velocity"],
            "active_frame_count": cw["active_frame_count"],
            "track_id_count": cw["track_id_count"],
        }

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Indices of the candidate windows that proceed to the AI handoff (and thus may
        become play_segments). The BASE sends ALL windows (no gating ? twins unchanged);
        ``FusionSegmenter`` overrides this to apply Gate A/B when thresholds are raised.
        Persisted candidates are NOT affected ? they remain the full superset for offline
        re-scoring."""
        return list(range(len(windows)))

    def process_video(self, video_id: str, video_path: str,  # type: ignore[override]
                      player_pool: Optional[List[str]] = None,
                      max_frames: Optional[int] = None) -> List[Dict[str, Any]]:
        # override-ignore: the ABC declares the Result contract (Success|Failure)
        # but every live segmenter still returns a bare list ? the documented
        # deliberate-incremental gap (CODE_MAP gotchas; main.py handles both).
        label = self.display_label
        # --- Sport profile + AI provider wiring (identical across segmenters) ---
        sport_name = self.config.get("sport", "badminton")
        try:
            sport_profile = sport_registry.get(sport_name)
        except KeyError as e:
            logger.error(str(e))
            return []

        idx_cfg = self.config.get("indexing", {})
        # Fully-local, NO-AI mode (default OFF): the trajectory candidate windows become the
        # rallies directly ? Phase 3's AI handoff is skipped, so no provider / API key is
```

```
        # needed and nothing is uploaded. Used to run the local detector standalone (e.g. to
        # measure it against the Gemini path, or to avoid the cloud entirely). With
        # FusionSegmenter the windows are still Gate-A/B-filtered via `_select_for_handoff`.
        skip_ai = bool(idx_cfg.get("skip_ai_handoff", False))
        provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider", "gemini"))
        if skip_ai:
            model_name = "local-noai"
            logger.info("%s in FULLY-LOCAL mode (skip_ai_handoff=True): candidate windows will "
                        "be emitted as rallies; no %s handoff, no API key needed.",
                        label, provider_name)
        else:
            model_name = self.config.get("ai_model", idx_cfg.get("ai_model", "gemini-2.5-flash"))
            api_key_env_var = f"{provider_name.upper()}_API_KEY"
            api_key = os.environ.get(api_key_env_var)
            if not api_key:
                from backend.pipeline.segmenters._keyhint import api_key_hint
                logger.error(
                    f"{api_key_env_var} environment variable is not set! "
                    f"To run the {provider_name} handoff, set your API key. "
                    + api_key_hint(api_key_env_var)
                )
                return []
            try:
                provider = provider_registry.get(provider_name, api_key=api_key, model_name=model_name)
            except KeyError as e:
                logger.error(str(e))
                return []

        video_duration = get_video_duration(video_path)
        if video_duration <= 0:
            logger.error("Invalid video duration.")
            return []
        fps, frame_width = self._probe_fps_width(video_path)

        # --- Runner config + windowing thresholds ---
        # _resolve_runner honours the CPU-stub override (RALLY_DETECTOR_IMPL /
        # indexing.detector_impl="stub") so the pipeline runs GPU/WSL-free; otherwise
        # it delegates to the subclass _build_runner (the real WSL/native runner).
        runner, runner_params = self._resolve_runner(idx_cfg)

        velocity_thresh = idx_cfg.get(self.family, {}).get("velocity_threshold", 0.02)
        merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
        min_window_duration = idx_cfg.get("min_action_window_duration", 2.0)
        # Bounded-windowing over-merge fix (W1): 0 = OFF (unchanged). See IndexingConfig.
        max_window_duration = idx_cfg.get("max_window_duration", 0.0)
        window_min_split_gap = idx_cfg.get("window_min_split_gap", 0.5)
        window_hard_cap = idx_cfg.get("window_hard_cap", False)
        window_inactivity_end = idx_cfg.get("window_inactivity_end", 0.0)
        inactivity_rally_thresh = idx_cfg.get("inactivity_rally_thresh", 0.08)
        inactivity_min_density = idx_cfg.get("inactivity_min_density", 0.34)
        window_blob_fallback = idx_cfg.get("window_blob_fallback", False)
        window_blob_factor = idx_cfg.get("window_blob_factor", 4.0)
        handoff_padding = idx_cfg.get("ai_handoff_padding", 2.0)
        ai_max_workers = idx_cfg.get("ai_max_workers", 5)
        log_candidates = idx_cfg.get("log_yolo_candidates", True)
        store_candidates = idx_cfg.get("store_yolo_candidates", True)

        detection_params = {
```

```
            "detector": self.name,
            "velocity_thresh": velocity_thresh,
            "merge_gap": merge_gap,
            "min_action_window_duration": min_window_duration,
            **runner_params,
        }

        # --- Phase 1: env healthcheck (fail fast with a clear message) ---
        ok, msg = runner.healthcheck()
        if not ok:
            logger.error("%s detector environment not ready: %s", label, msg)
            return []
        logger.info("%s detector env OK: %s", label, msg)

        # --- Phase 2: trajectory -> candidate rally windows ---
        # Trajectory detectors process the whole video in one pass (they carry
        # their own frame buffering), so unlike yolo_hybrid we don't pre-chunk.
        t_start = time.time()
        out_dir = os.path.join("output", self.family)
        csv_path = runner.run_predict(video_path, out_dir, video_id=video_id)
        if not csv_path:
            logger.error("%s produced no trajectory; aborting.", label)
            return []

        points = runner.parse_trajectory_csv(csv_path)
        logger.info("Parsed %d trajectory points (%d visible).",
                    len(points), sum(1 for p in points if p.visible))

        all_candidate_windows = runner.trajectory_to_action_windows(
            points, fps=fps, frame_width=frame_width, chunk_start=0.0,
            velocity_thresh=velocity_thresh, merge_gap=merge_gap,
            min_window_duration=min_window_duration, chunk_index=0,
            max_window_duration=max_window_duration, min_split_gap=window_min_split_gap,
            window_hard_cap=window_hard_cap,
            window_inactivity_end=window_inactivity_end,
            inactivity_rally_thresh=inactivity_rally_thresh,
            inactivity_min_density=inactivity_min_density,
            window_blob_fallback=window_blob_fallback,
            window_blob_factor=window_blob_factor,
        )
        logger.info("Phase 2 (%s) completed in %.1fs. Candidate windows: %d",
                    label, time.time() - t_start, len(all_candidate_windows))

        if log_candidates:
            for cw in all_candidate_windows:
                logger.info(f"[{label} rally] {_fmt_ts(cw['start'])}-{_fmt_ts(cw['end'])} "
                            f"({cw['end'] - cw['start']:.1f}s) | frames={cw['active_frame_count']} "
                            f"peak_v={cw['peak_velocity']:.3f} mean_v={cw['mean_velocity']:.3f}")

        # Per-window stats via the enrichment hook ? base = the 4 motion keys; the fusion
        # segmenter adds net_crossings + person-cue features. Computed once, reused for both
        # persistence and the handoff gate below.
        window_stats = [
            self._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
            for cw in all_candidate_windows
        ]

        # Persist raw candidates for the fine-tuning dataset (same table as YOLO).
        candidate_ids: List[Optional[int]] = [None] * len(all_candidate_windows)
```

```
        if store_candidates:
            for i, cw in enumerate(all_candidate_windows):
                candidate_ids[i] = self.db.add_candidate_segment(
                    video_id, detector=self.name,
                    start_time=cw["start"], end_time=cw["end"],
                    chunk_index=cw["chunk_index"], confidence=min(1.0, cw["peak_velocity"]),
                    stats=window_stats[i], detection_params=detection_params,
                )

        if not all_candidate_windows:
            return []

        # --- Phase 3: AI provider handoff over padded candidate clips ---
        # Which windows reach the handoff (base = all; fusion may apply Gate A/B). Persisted
        # candidates above stay the full superset ? only the served play_segments are gated.
        handoff_indices = self._select_for_handoff(all_candidate_windows, window_stats, idx_cfg)

        ai_segments: List[dict] = []
        if skip_ai:
            # Fully-local: the selected candidate windows ARE the rallies ? emit them verbatim
            # (no clip extraction, no upload). Boundaries are the local detector's; shot count
            # falls back to the duration-based estimate (no AI exchange count).
            ai_segments = [
                {
                    "start_time": all_candidate_windows[wi]["start"],
                    "end_time": all_candidate_windows[wi]["end"],
                    "shuttle_exchange_count": None,
                    "shot_count_confidence": "LocalNoAI",
                    "_candidate_id": candidate_ids[wi],
                }
                for wi in handoff_indices
            ]
            logger.info("Phase 3 SKIPPED (fully-local): emitted %d candidate windows as rallies "
                        "(no AI handoff).", len(ai_segments))
        else:
            handoff_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
            os.makedirs(handoff_dir, exist_ok=True)
            logger.info("Phase 3: %s validation on %d candidate windows...",
                        provider_name, len(handoff_indices))

            def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
                w_start = max(0, window["start"] - handoff_padding)
                w_end = min(video_duration, window["end"] + handoff_padding)
                w_dur = w_end - w_start
                w_path = os.path.join(handoff_dir, f"handoff_{video_id}_{wi}.mp4")
                if not extract_video_chunk(video_path, w_start, w_dur, w_path, reencode=True):
                    return []
                prompt = sport_profile.get_prompt(chunk_duration=w_dur)
                segments, _ = provider.analyze_video(w_path, prompt, chunk_duration=w_dur,
                                                     label=f"Handoff {wi+1}")
                valid = []
                for seg in segments:
                    try:
                        seg["start_time"] = float(seg["start_time"]) + w_start
                        seg["end_time"] = float(seg["end_time"]) + w_start
                        seg["_candidate_id"] = candidate_ids[wi]
```

```
                        valid.append(seg)
                    except (ValueError, TypeError, KeyError) as e:
                        logger.warning("Dropping segment with unparseable timestamps: %s ? %s", seg, e)
                try:
                    os.remove(w_path)
                except OSError:
                    pass
                return valid

            with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as executor:
                futures = [executor.submit(process_candidate_window, wi, all_candidate_windows[wi])
                           for wi in handoff_indices]
                for future in futures:
                    ai_segments.extend(future.result())
            try:
                os.rmdir(handoff_dir)
            except OSError:
                pass
            logger.info("Phase 3 completed. AI confirmed %d semantic play segments.", len(ai_segments))

        # --- Phase 4: DB population + candidate linking ---
        segments_data = []
        for segment in ai_segments:
            try:
                start = float(segment["start_time"])
                end = float(segment["end_time"])
                dur = end - start
                metadata = {
                    "shot_count": self._shot_count_from(segment, dur),
                    "shot_count_confidence": segment.get("shot_count_confidence", "Unknown"),
                    "detector": f"{self.family}-hybrid-{model_name}",
                }
                segment_id = self.db.add_play_segment(video_id, start, end, None, None, metadata)
                if segment_id:
                    candidate_id = segment.get("_candidate_id")
                    if candidate_id is not None:
                        self.db.link_candidate_to_segment(candidate_id, segment_id)
                    segments_data.append({
                        "id": segment_id, "start_time": start, "end_time": end,
                        "duration": dur, "metadata": metadata,
                    })
            except Exception as e:
                logger.warning("Skipping invalid segment struct: %s", e)

        return segments_data

```

9. assistant (2026-06-20T08:57:25.708Z)

Now let me read the wasb_hybrid and fusion_hybrid overrides.

10. user (2026-06-20T08:57:26.661Z)

```
"""WASB-based hybrid play segmenter.

Phase 2 uses **WASB** (nttcom/WASB-SBDT, MIT) shuttle-trajectory detection via
WSL2 (backend/pipeline/detectors/wasb_runner.py). WASB ships pretrained
badminton weights and runs today, so this is the live shuttle-motion substrate.

```

```
For the per-video *calibration* goal we mainly use Phase 2 (trajectory ->
candidate windows); the windowing thresholds (velocity_thresh / merge_gap /
min_window_duration) are what get tuned against golden rally labels.

The pipeline engine (healthcheck -> trajectory -> windows -> AI handoff -> DB)
lives in trajectory_hybrid.TrajectoryHybridSegmenter; this class only wires the
WASB runner and identity.
"""
from typing import Any, Dict, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig


class WasbHybridSegmenter(TrajectoryHybridSegmenter):
    family = "wasb"
    display_label = "WASB"

    @property
    def name(self) -> str:
        return "wasb_hybrid"

    @property
    def description(self) -> str:
        return ("Two-stage pipeline: WASB shuttle-trajectory detection (WSL2, pretrained badminton "
                "weights) for precise candidate rallies + AI provider for semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        cfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(cfg), {"weights_path": cfg.weights_path}

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        # Lazy import: the native runner pulls no heavy deps, but keep parity with the stub
        # seam (imported only when detector_impl='native' is actually selected).
        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner, NativeWasbConfig
        cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
        return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path,
                                       "detector_impl": "native", "device": cfg.device}


segmenter_registry.register(WasbHybridSegmenter)

```

11. user (2026-06-20T08:57:27.129Z)

```
"""Multi-signal fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2).

Shuttle stays PRIMARY: this reuses an existing trajectory substrate (WASB by default,
TrackNet optional) to seed candidate windows exactly as `wasb_hybrid`/`tracknet_hybrid`
do ? it subclasses the same `TrajectoryHybridSegmenter` engine. It then ENRICHES each
shuttle-seeded window via two overridable seams the engine exposes:

- ``_enrich_candidate_stats`` ? always adds the **net-crossing count** (Gate A signal,
  GPU-free, computed from the trajectory we already have); and, only when the person cue
  is explicitly enabled (``indexing.fusion.cue_flags.person``), the person-cue features
  (`fusion_features`), persisting them into ``candidate_segments.stats`` for the learned
  fusion (P3). The person path lazily builds ONE `PersonDetector` and only then loads
```

```
the
  torchvision/supervision model ? so the default (person-OFF) path never loads a detector
  model nor touches the GPU. (torch/cv2 themselves are imported by the registry regardless,
  via the pre-existing yolo_hybrid ? person_detector chain; the lazy part is the model.)
- ``_select_for_handoff`` ? optional served Gate A/B: when ``indexing.fusion.min_crossings``
  (and, with the person cue, ``min_players``) are raised above 0, sub-threshold windows are
  dropped from the AI handoff (the held-shuttle / one-sided-feed false-positive fix).
  **Persisted candidates remain the full superset** regardless, so the offline experiment
  harness can re-score any variant.

Defaults reproduce the substrate exactly: net_crossings/person features are persisted but
nothing is gated (thresholds 0, person OFF), so `FusionSegmenter` with default config seeds
and hands off identically to `wasb_hybrid`. Registered as ``fusion_hybrid``, NOT the default
segmenter ? it is opt-in. Promotion to default happens only on measured LOVO lift through the
experiment harness (EXPERIMENT_HARNESS.md), never silently.
"""
from typing import Any, Dict, List, Optional, Tuple

from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
from backend.pipeline.detectors.base import DetectorRunner
from backend.pipeline.detectors.wasb_runner import WasbRunner, WasbConfig
from backend.pipeline.detectors.tracknet_runner import TrackNetRunner, TrackNetConfig
from backend.pipeline.detectors import rally_gate
from backend.pipeline.detectors import fusion_features as ff


class FusionSegmenter(TrajectoryHybridSegmenter):
    """Shuttle trajectory (primary) + net-crossing Gate A + optional person cue (Gate B)."""

    family = "fusion"
    display_label = "Fusion"

    def __init__(self, db: Any, config: Any):
        super().__init__(db, config)
        # Built lazily only if the person cue is enabled (no detector model loaded otherwise).
        self._person: Optional[Any] = None
        # Cache the play-axis/net estimate per trajectory so rally_gate doesn't re-estimate it
        # over the whole trajectory once per candidate window (it depends only on `points`).
        self._axis_cache_key: Optional[int] = None
        self._axis_cache: Optional[tuple] = None

    @property
    def name(self) -> str:
        return "fusion_hybrid"

    @property
    def description(self) -> str:
        return ("Multi-signal fusion: shuttle trajectory (WASB/TrackNet) seeds candidate rallies, "
                "net-crossing Gate A + optional person-occupancy Gate B adjudicate them, and the "
```

```
                "AI provider sets semantic boundaries.")

    def _build_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
        if substrate == "tracknet":
            cfg = TrackNetConfig.from_indexing_cfg(idx_cfg)
            return TrackNetRunner(cfg), {
                "weights_path": cfg.weights_path,
                "queue_length": cfg.queue_length,
                "fusion_substrate": "tracknet",
            }
        wcfg = WasbConfig.from_indexing_cfg(idx_cfg)
        return WasbRunner(wcfg), {"weights_path": wcfg.weights_path, "fusion_substrate": "wasb"}

    def _build_native_runner(self, idx_cfg: Dict[str, Any]) -> Tuple[DetectorRunner, Dict[str, Any]]:
        # Native (no-WSL) path supports the WASB substrate only ? there is no native TrackNet
        # runner yet. Fusion's default substrate is WASB, so this covers the common case.
        substrate = idx_cfg.get("fusion", {}).get("substrate", "wasb")
        if substrate != "wasb":
            raise NotImplementedError(
                "detector_impl='native' supports the WASB substrate only ? set "
                "indexing.fusion.substrate='wasb' (no native TrackNet runner yet).")
        from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner, NativeWasbConfig
        cfg = NativeWasbConfig.from_indexing_cfg(idx_cfg)
        return NativeWasbRunner(cfg), {"weights_path": cfg.weights_path, "detector_impl": "native",
                                       "fusion_substrate": "wasb", "device": cfg.device}

    def _enrich_candidate_stats(self, cw: Dict[str, Any], points: List[Any], fps: float,
                                frame_width: float, video_path: str,
                                idx_cfg: Dict[str, Any]) -> Dict[str, Any]:
        stats = super()._enrich_candidate_stats(cw, points, fps, frame_width, video_path, idx_cfg)
        # Gate-A signal ? net-crossing count for this window. Pure, GPU-free, always persisted
        # (single service feed crosses ~once; a real rally crosses >=2x). axis/net auto-estimated
        # over the whole trajectory by rally_gate (camera-agnostic); reused across windows.
        gated = rally_gate.gate_windows(points, [(cw["start"], cw["end"])], fps,
                                        min_crossings=0, estimate=self._axis_estimate(points))
        stats["net_crossings"] = float(gated[0].crossings) if gated else 0.0

        fcfg = idx_cfg.get("fusion", {})
        if fcfg.get("cue_flags", {}).get("person", False):
            stats.update(self._person_features(cw, points, fps, frame_width, video_path, fcfg))
        return stats

    # P3 (learned fusion) first-step landing note (per MULTI_SIGNAL_FUSION_PLAN + NEXT_STEPS 2026-06-14):
    # Person features (from _person_features when cue_flags.person) are now persisted for the harness
    # `compare` LOVO litmus on the 6 golden videos. Eager-person-compute optimisation (compute only
    # for shuttle-seeded windows + padding, not whole video) is already in place via the lazy
    # self._person + windowed detections_per_frame. Next: actual harness run + lift
```

```
measurement
    # (owner/hardware) before any served promotion. Flag-gated (default person OFF). See
    # docs/NEXT_STEPS.md "Rally-detection quality track" and EXPERIMENT_HARNESS.md for discipline.

    def _axis_estimate(self, points: List[Any]) -> tuple:
        """Cache rally_gate's (axis, net, span) play-axis estimate per trajectory so it's
        computed once per video, not once per candidate window (it depends only on points)."""
        key = id(points)
        if self._axis_cache_key != key or self._axis_cache is None:
            self._axis_cache_key = key
            self._axis_cache = rally_gate.estimate_play_axis(points)
        est = self._axis_cache
        assert est is not None
        return est

    def _person_features(self, cw: Dict[str, Any], points: List[Any], fps: float,
                         frame_width: float, video_path: str,
                         fcfg: Dict[str, Any]) -> Dict[str, float]:
        """GPU path (person cue ON): run the person detector over this window's ORIGINAL-video
        frame range and compute the scale/translation-invariant person features. Lazily builds
        ONE PersonDetector. ? real-video verification is owner-side; this wiring is mock-tested."""
        # Lazy import so the default (person-OFF) path never pulls torch/cv2 through this module.
        from backend.pipeline.detectors.person_detector import PersonDetector
        if self._person is None:
            self._person = PersonDetector(self.config)

        frame_skip = self.config.get("indexing", {}).get("yolo_frame_skip", 3)
        # round (not truncate): int() would map e.g. 1.999s*30 to frame 59 not 60 ? a
        # window-edge off-by-one that drops the boundary frame from the shuttle alignment.
        start_f, end_f = round(cw["start"] * fps), round(cw["end"] * fps)
        det = self._person.detections_per_frame(video_path, start_f, end_f, frame_skip)
        fw = det["frame_width"] or frame_width
        fh = det["frame_height"]
        # If we can't trust the frame dims, every normalised feature would silently collapse
        # to ~0 ? return an explicit all-zero person-feature row instead of a half-baked one.
        if fw <= 0 or fh <= 0:
            return {k: 0.0 for k in ff.PERSON_FEATURE_NAMES}

        poly = fcfg.get("court_polygon")
        polygon = [(float(p[0]), float(p[1])) for p in poly] if poly else None
        net = (fcfg.get("net_axis", "y"), float(fcfg.get("net_position", 0.5)))
        shuttle_in = [p for p in points if start_f <= p.frame <= end_f]
        return ff.compute_person_features(det["frames"], shuttle_in, fw, fh,
                                          court_polygon=polygon, net=net)

    def _select_for_handoff(self, windows: List[Dict[str, Any]],
                            window_stats: List[Dict[str, Any]],
                            idx_cfg: Dict[str, Any]) -> List[int]:
        """Served Gate A/B over the (already-enriched) per-window stats. Default thresholds
        (0) keep every window ? identical to the substrate. Persisted candidates are
        untouched (full superset).

        NEVER-EMPTY safeguard: a video that had ?1 window must keep ?1 after gating. If the
```

```
        thresholds would drop EVERY window (e.g. a short, noisy or one-sided-feed trajectory
        where no segment clears ``min_crossings``), the gate falls back to the full ungated
        set rather than emit a video with zero rallies. This makes Gate A/B strictly
        *pruning* ? it can shrink a candidate set but never zero it out ? so layering it on
        the substrate is "do no harm": the bounded-windowing / largest-internal-gap split
        below it always wins on an otherwise-empty video. Observed on real footage
        (mahadevpura-1): a single mega-window split into pieces, none clearing the crossings
        gate ? without this guard the whole clip yielded 0 rallies (no reel)."""
        fcfg = idx_cfg.get("fusion", {})
        min_crossings = int(fcfg.get("min_crossings", 0))
        min_players = int(fcfg.get("min_players", 0))
        person_on = bool(fcfg.get("cue_flags", {}).get("person", False))

        keep: List[int] = []
        for i, st in enumerate(window_stats):
            if min_crossings > 0 and st.get("net_crossings", 0.0) < min_crossings:
                continue
            if person_on and min_players > 0 and st.get("players_in_court", 0.0) < min_players:
                continue
            keep.append(i)

        if not keep and window_stats:
            # Gating emptied a non-empty video ? fall back to the ungated set.
            return list(range(len(window_stats)))
        return keep


segmenter_registry.register(FusionSegmenter)

```

12. assistant (2026-06-20T08:57:32.822Z)

Now let me focus on the key concern: does `WasbIndexConfig.device`/`python_bin` survive `model_dump` so `NativeWasbConfig.from_indexing_cfg` sees them? The critical path is how `idx_cfg` is constructed. Let me trace `self.config.get("indexing", {})`.

Looking at `AppConfig.get`: when called with `"indexing"`, it returns `val.model_dump(mode="json")`. So the `wasb` sub-section gets dumped too. Let me verify the nested dump includes `device`/`python_bin` and that `from_indexing_cfg` reads `idx_cfg["wasb"]` (a dict). Let me check the WSL `WasbConfig.from_indexing_cfg` for comparison, and check how `from_indexing_cfg` is called regarding env precedence.

13. user (2026-06-20T08:57:33.504Z)

```
"""WASB shuttle-trajectory runner (WSL2 subprocess adapter).

Windows-side counterpart to backend/detectors/wasb_infer.py. It:
  1. stages the video into the WSL filesystem (fast local reads, not /mnt),
  2. copies wasb_infer.py into the WASB repo's src/ dir,
  3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
  4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.

So the only WASB-specific bits live here; the windowing + CSV parsing are shared
with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
"""
import os
import shlex
import logging
import subprocess
```

```python
from dataclasses import dataclass
from typing import Dict, Any, Optional, Tuple

# Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
# parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on TrackNetRunner.
from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin, wsl_tilde_quote

logger = logging.getLogger(__name__)

# Windows path to the inference wrapper we stage into WSL.
_INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")


@dataclass
class WasbConfig:
    """Resolved settings for a WASB WSL run (built from config.json -> indexing.wasb)."""
    distro: str = "Ubuntu"
    repo_dir: str = "~/models/WASB-SBDT"
    conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
    conda_env: str = "wasb"
    weights_path: str = "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
    sport: str = "badminton"
    wsl_stage_dir: str = "~/clips"
    timeout_sec: int = 1800  # 30 min; a hung GPU/conda call is killed (0 = no timeout)
    keep_frames: bool = False  # retain staged video + frame cache after success (debug only)
    min_free_gb: float = 0.0   # warn if WSL free space below this before a run (0 = off)
    # FAST PATH (default OFF until verified): decode the staged video on the fly inside
    # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
    # extraction that dominates a long clip. Output is bit-identical to the PNG path
    # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms it.
    stream_video: bool = False

    @classmethod
    def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
        w = (idx_cfg or {}).get("wasb", {}) or {}
        return cls(
            distro=w.get("wsl_distro", "Ubuntu"),
            repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
            conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
            conda_env=w.get("conda_env", "wasb"),
            weights_path=w.get("weights_path",
                                   "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
            sport=w.get("sport", "badminton"),
            wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
            timeout_sec=int(w.get("timeout_sec", 1800)),
            keep_frames=bool(w.get("keep_frames", False)),
            min_free_gb=float(w.get("min_free_gb", 0.0)),
            stream_video=bool(w.get("stream_video", False)),
        )


class WasbRunner(WslCommandMixin, DetectorRunner):
    def __init__(self, cfg: WasbConfig):
        self.cfg = cfg

    def healthcheck(self) -> Tuple[bool, str]:
```

```
        """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
        cmd = (f"conda activate {self.cfg.conda_env} && "
               f"python -c \"import torch; print('torch', torch.__version__, "
               f"'cuda', torch.cuda.is_available())\"")
        try:
            res = self._wsl_bash(cmd)
        except FileNotFoundError:
            return False, "`wsl` not found ? Windows + WSL2 required."
        except subprocess.TimeoutExpired:
            return False, "WSL healthcheck timed out."
        out = (res.stdout or "").strip()
        if res.returncode != 0:
            return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
        return True, out

    def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
        src_mnt = to_wsl_mnt_path(video_win_path)
        reason = self.wsl_source_unreadable_reason(src_mnt)
        if reason:
            logger.error("Cannot stage video into WSL: %s", reason)
            return None
        dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
        cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} {wsl_tilde_quote(dest)} && echo OK"
        try:
            res = self._wsl_bash(cmd)
        except subprocess.TimeoutExpired:
            logger.error("Staging video into WSL timed out.")
            return None
        if res.returncode != 0 or "OK" not in (res.stdout or ""):
            logger.error("Failed to stage video into WSL: %s", (res.stderr or res.stdout).strip())
            return None
        return dest

    def _stage_infer_script(self) -> bool:
        """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
        src_mnt = to_wsl_mnt_path(_INFER_WIN)
        cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo OK"
        res = self._wsl_bash(cmd)
        return res.returncode == 0 and "OK" in (res.stdout or "")

    def run_predict(self, video_win_path: str, output_win_dir: str,
                    video_id: Optional[str] = None) -> Optional[str]:
        """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.

        All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
        two videos that share a filename (e.g. two ``match.mp4`` from different folders)
        can never reuse each other's staged frames, resumable cache, or CSV.
        """
        # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
        # translation: to_wsl_mnt_path passes relative paths through unchanged, so
        # WSL resolved them against the inference cwd (~/models/WASB-SBDT/src) and
        # the CSV landed inside WSL while Windows polled the repo-relative path.
        output_win_dir = os.path.abspath(output_win_dir)
        os.makedirs(output_win_dir, exist_ok=True)
        # Normalize for cross-platform basename (tests use Windows paths on Linux).
        video_win_path = str(video_win_path).replace("\\", "/")
        base = os.path.basename(video_win_path)
        stem, ext = os.path.splitext(base)
        # Unique, content-derived key: same filename + different bytes -> different key.
        key = f"{stem}__{video_id[:12]}" if video_id else stem
        wsl_out_dir = to_wsl_mnt_path(output_win_dir)
        expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
```

```python

        if not self._stage_infer_script():
            logger.error("Could not stage wasb_infer.py into WSL.")
            return None
        staged_video = self._stage_video(video_win_path, f"{key}{ext}")
        if staged_video is None:
            return None
        frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"

        # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
        # WSL stage filesystem is already low so the user can clean up before it fills.
        # (Streaming never materializes the PNGs, so the guard only matters off the fast path.)
        if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
            free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
            if free is not None and free < self.cfg.min_free_gb:
                logger.warning(
                    "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame extraction "
                    "may fill the disk ? consider running tools/cleanup_caches.py.",
                    free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)

        # Fast path: stream-decode the video in-process (no PNG extraction). Output is
        # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min clip.
        if self.cfg.stream_video:
            frame_args = "--stream-video "
        else:
            frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
        cmd = (
            f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
            f"python wasb_infer.py "
            f"--video {wsl_tilde_quote(staged_video)} "
            f"{frame_args}"
            f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
            f"--sport {shlex.quote(self.cfg.sport)} "
            f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
        )
        logger.info("Running WASB inference on %s ...", base)
        try:
            res = self._wsl_bash(cmd)
        except subprocess.TimeoutExpired:
            logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
            return None
        if res.returncode != 0:
            logger.error("WASB inference failed:\n%s", (res.stderr or res.stdout)[-2000:])
            return None
        if not os.path.exists(expected_csv_win):
            logger.error("WASB finished but expected CSV not found at %s", expected_csv_win)
            return None
        logger.info("WASB trajectory CSV: %s", expected_csv_win)

        # Success ? the trajectory CSV is the durable output; the staged video copy and
        # the (huge) decoded frame cache are now pure intermediates. Delete them unless
        # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
        # so the frame cache + detector cache remain for a resumable re-run.
        # (Streaming never writes a frame cache, so there's only the staged video to drop.)
        if not self.cfg.keep_frames:
            stale = [staged_video] if self.cfg.stream_video else [frames_dir, staged_video]
            logger.info("Cache hygiene: removing staged video%s for %s "
                        "(set indexing.wasb.keep_frames=true to retain).",
```

```
                        "" if self.cfg.stream_video else " + frame cache", key)
            self._wsl_rm_rf(stale)
        return expected_csv_win

    # Trajectory parsing + windowing are inherited from tracknet_runner (model-agnostic).
    parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
    trajectory_to_action_windows = staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

14. assistant (2026-06-20T08:57:36.865Z)

Now I notice the WSL `WasbConfig.from_indexing_cfg` does NOT read env vars, while the native one does. Let me note that the env-precedence behavior is unique to the native config. Now let me check the worker `__main__.py` and the tests.

15. user (2026-06-20T08:57:37.284Z)

```
"""Worker dispatcher ? pull from storage ? run the pipeline ? push outputs back.

    python -m backend.worker infer --video-uri <uri> --out-uri <prefix> \
        [--segmenter wasb_hybrid] [--config config.json] [--set k=v ...] [--scratch DIR]
    python -m backend.worker train ...       # Gen-0 train-from-bucket (follow-up)

The URI scheme selects the storage backend via ``backend.storage`` (a bare path / ``local://``
is an identity passthrough; ``s3://`` / ``gcs://`` is an object store), so existing local runs
are byte-for-byte unchanged. Combine with ``RALLY_DETECTOR_IMPL=stub`` and
``--set indexing.skip_ai_handoff=true`` for a **secret-free, GPU-free** run (no Gemini key) ?
the path validated end-to-end on DigitalOcean (``docs/SETUP_NEW_MACHINE.md``).

This is a thin orchestrator: it composes the SAME building blocks the ``backend.main`` CLI uses
(validator registry ? segmenter ? report), never a forked pipeline.
"""
from __future__ import annotations

import argparse
import csv
import json
import os
import sys
import tempfile
from typing import Any, List

from backend import storage
from backend.config import load_config
from backend.infrastructure.database import Database
from backend.pipeline.segmenters.base import segmenter_registry
from backend.pipeline.validators.base import registry as validator_registry
from backend.reporting import emit_indexer_report
from backend.results import Failure
from backend.utils.hashing import compute_video_id


def _coerce(v: str) -> Any:
    """Coerce a ``--set`` string value to bool/int/float where it round-trips, else str."""
    low = v.lower()
    if low in ("true", "false"):
        return low == "true"
    for cast in (int, float):
```

```
        try:
            return cast(v)
        except ValueError:
            pass
    return v


def _apply_overrides(config: Any, sets: List[str]) -> None:
    """Apply ``a.b.c=value`` overrides onto the typed config (e.g. indexing.skip_ai_handoff=true)."""
    for kv in sets or []:
        key, sep, val = kv.partition("=")
        if not sep:
            raise ValueError(f"--set expects k=v, got {kv!r}")
        parts = key.split(".")
        obj = config
        for p in parts[:-1]:
            obj = getattr(obj, p)
        setattr(obj, parts[-1], _coerce(val))


def _write_intervals_csv(segments: List[dict], path: str) -> None:
    """Decimal-second ``[start,end)`` rallies + shot count / ending reason ? the join-friendly
    artifact (same schema as the rally-annotator labels)."""
    with open(path, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["start", "end", "shot_count", "ending_reason"])
        for s in segments:
            w.writerow([
                f'{float(s.get("start_time", 0.0)):.3f}',
                f'{float(s.get("end_time", 0.0)):.3f}',
                s.get("shot_count", ""),
                s.get("ending_reason", s.get("shot_count_confidence", "")),
            ])


def _write_report_json(video_id: str, segments: List[dict], status: str, path: str) -> None:
    with open(path, "w") as f:
        json.dump({
            "video_id": video_id,
            "status": status,
            "segment_count": len(segments),
            "segments": [{"start": float(s.get("start_time", 0.0)),
                          "end": float(s.get("end_time", 0.0))} for s in segments],
        }, f, indent=2)


def cmd_infer(args: argparse.Namespace) -> int:
    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    storage_cfg = config.storage.model_dump()

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-worker-")
    os.makedirs(scratch, exist_ok=True)
    config.output.output_dir = scratch

    # 1. PULL ? local:// / bare path = identity passthrough; s3:// / gcs:// = download to scratch.
    local_video = storage.fetch(args.video_uri, os.path.join(scratch, "in.mp4"), config=storage_cfg)
    print(f"[worker] pulled {args.video_uri} -> {local_video}")

    # 2. IDENTITY ? whole-file MD5 (the labels?video join key; pulled bytes must be
```

```
byte-identical).
    video_id = compute_video_id(local_video)

    # 3. VALIDATE (same registry as the main CLI).
    val_results, val_context = validator_registry.run_all_with_context(local_video, config)
    failures = [r for r in val_results if not r.passed]
    db = Database(os.path.join(scratch, config.output.db_name))
    db.add_video(video_id, local_video, "failed" if failures else "processed",
                 [r.model_dump() for r in val_results])

    segments: List[dict] = []
    segmenter_actual = None
    segmentation_ran = False
    status = "failed"
    try:
        if failures:
            print("[worker] validation FAILED: "
                  + "; ".join(f"{r.validator_name}: {r.message}" for r in failures))
            return 2
        seg_name = args.segmenter or config.indexing.default_segmenter
        segmenter_cls = segmenter_registry.get(seg_name)
        if not segmenter_cls:
            print(f"[worker] unknown segmenter: {seg_name!r} "
                  f"(have {list(segmenter_registry.list_available())})")
            return 2
        segmenter_actual = seg_name
        result = segmenter_cls(db, config).process_video(video_id, local_video, max_frames=args.max_frames)
        segmentation_ran = True
        if isinstance(result, Failure):
            print(f"[worker] segmenter FAILED: {result.message}")
            return 3
        segments = result.value if hasattr(result, "value") else result
        status = "success"
    finally:
        # Best-effort per-video observability report (never raises) ? parity with the main CLI.
        emit_indexer_report(
            db=db, video_id=video_id, video_path=local_video, config=config,
            val_results=val_results, val_context=val_context,
            segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
            was_reingest=False, segmentation_ran=segmentation_ran,
        )

    # 5. SERIALIZE + 6. PUSH (outputs/<video_id>/?). idempotent on video_id.
    out_local = os.path.join(scratch, "outputs", video_id)
    os.makedirs(out_local, exist_ok=True)
    _write_intervals_csv(segments, os.path.join(out_local, "intervals.csv"))
    _write_report_json(video_id, segments, status, os.path.join(out_local, "report.json"))

    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(out_local)):
        uri = f"{out_prefix}/outputs/{video_id}/{fn}"
        storage.put(os.path.join(out_local, fn), uri, config=storage_cfg)
        pushed.append(uri)

    print(f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0

```

```
def _resolve_train_detector(args: argparse.Namespace, config: Any):
    """Build the trajectory DetectorRunner for `--trajectory-source detector`, honouring
    detector_impl (RALLY_DETECTOR_IMPL / indexing.detector_impl: auto=WSL, native=GPU box,
    stub=CPU mock). Returns the runner, or prints an error + returns None.

    Reuses the segmenter's `_resolve_runner` seam so the worker and the live CLI build the
    detector identically (one source of truth). A non-trajectory segmenter (e.g. yolo) has
    no shuttle detector and is rejected.
    """
    from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter

    seg_cls = segmenter_registry.get(args.segmenter)
    if not seg_cls:
        print(f"[worker] unknown segmenter: {args.segmenter!r} "
              f"(have {list(segmenter_registry.list_available())})")
        return None
    seg = seg_cls(None, config)
    if not isinstance(seg, TrajectoryHybridSegmenter):
        print(f"[worker] --segmenter {args.segmenter!r} has no shuttle detector; "
              f"use a trajectory hybrid (wasb_hybrid / tracknet_hybrid / fusion_hybrid).")
        return None
    runner, _ = seg._resolve_runner(config.get("indexing", {}))
    ok, msg = runner.healthcheck()
    if not ok:
        print(f"[worker] detector environment not ready: {msg}")
        return None
    print(f"[worker] detector ready ({args.segmenter}): {msg}")
    return runner


def cmd_train(args: argparse.Namespace) -> int:
    """Gen-0 train-from-bucket: pull labels (+ videos) -> trajectories -> manifest -> shell out
    to training.gen0.harness (backend must NOT import training) -> push the artifact bundle.

    Two trajectory sources (the train pipeline + harness are identical either way):
      * ``synth`` (default) ? the CPU "mock GPU": deterministic, label-consistent synthetic
        shuttle paths (no GPU/WSL), so the whole pipeline is validated on a CPU box / CI.
      * ``detector`` ? REAL shuttle trajectories: pull each video and run the resolved
        DetectorRunner (native WASB on a GPU box, or stub for a GPU-free wiring test). This
        is the M3.5 "first real training" path; only the trajectory source flips.
    """
    import subprocess

    config = load_config(args.config) if args.config else load_config()
    _apply_overrides(config, args.set)
    storage_cfg = config.storage.model_dump()

    scratch = args.scratch or tempfile.mkdtemp(prefix="rally-train-")
    labels_dir = os.path.join(scratch, "labels")
    traj_dir = os.path.join(scratch, "trajectories")
    videos_dir = os.path.join(scratch, "videos")
    os.makedirs(labels_dir, exist_ok=True)
    os.makedirs(traj_dir, exist_ok=True)

    corpus = args.corpus_uri.rstrip("/")
```

```
        label_uris = sorted(u for u in storage.list_uris(f"{corpus}/labels/", config=storage_cfg)
                            if u.lower().endswith(".csv"))
        if len(label_uris) < 2:
            print(f"[worker] need >=2 labeled videos for leave-one-video-out; found {len(label_uris)} "
                  f"under {corpus}/labels/")
            return 2

        # Real-detector source: resolve the runner once + index the bucket's videos/ by stem.
        runner = None
        video_by_stem: dict = {}
        if args.trajectory_source == "detector":
            os.makedirs(videos_dir, exist_ok=True)
            runner = _resolve_train_detector(args, config)
            if runner is None:
                return 2
            for vuri in storage.list_uris(f"{corpus}/videos/", config=storage_cfg):
                vname = storage.parse_uri(vuri).name
                video_by_stem[os.path.splitext(vname)[0]] = vuri

        print(f"[worker] trajectory source: {args.trajectory_source}")
        specs: List[dict] = []
        for uri in label_uris:
            fname = storage.parse_uri(uri).name                    # e.g. GX020094_proxy.rallies.csv
            name = fname.replace(".rallies.csv", "").replace(".csv", "")
            local_label = storage.fetch(uri, os.path.join(labels_dir, fname), config=storage_cfg)
            if args.trajectory_source == "detector":
                vuri = video_by_stem.get(name)
                if not vuri:
                    print(f"[worker] no video for {name!r} under {corpus}/videos/ ? skipping.")
                    continue
                local_video = storage.fetch(vuri, os.path.join(videos_dir, storage.parse_uri(vuri).name),
                                            config=storage_cfg)
                video_id = compute_video_id(local_video)
                # Real fps/width drive the trajectory frame->time mapping (synth fixes them at 30/1280).
                from backend.pipeline.segmenters.trajectory_hybrid import TrajectoryHybridSegmenter
                fps, frame_width = TrajectoryHybridSegmenter._probe_fps_width(local_video)
                traj = runner.run_predict(local_video, traj_dir, video_id=video_id)
                if not traj:
                    print(f"[worker] detector produced no trajectory for {name!r} ? skipping.")
                    continue
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": fps, "frame_width": frame_width})
            else:
                from backend.pipeline.detectors.stub_runner import synth_trajectory_from_labels
                traj = os.path.join(traj_dir, f"{name}_traj.csv")
                synth_trajectory_from_labels(local_label, traj, fps=args.fps, frame_width=args.frame_width)
                specs.append({"name": name, "trajectory": traj, "labels": local_label,
                              "fps": args.fps, "frame_width": args.frame_width})

        if len(specs) < 2:
            print(f"[worker] need >=2 videos with trajectories for leave-one-video-out; got {len(specs)}.")
            return 2
```

```
    manifest_path = os.path.join(scratch, "manifest.json")
    with open(manifest_path, "w", encoding="utf-8") as f:
        json.dump(specs, f, indent=2)
    src_label = "real detector" if args.trajectory_source == "detector" else "CPU-mock stub"
    print(f"[worker] built manifest: {len(specs)} videos ({src_label} trajectories)")

    out_dir = os.path.join(scratch, "artifacts")
    cmd = [sys.executable, "-m", "training.gen0.harness",
           "--manifest", manifest_path, "--out", out_dir, "--version", args.version]
    if args.floor:
        floor_local = (storage.fetch(args.floor, os.path.join(scratch, "floor.json"), config=storage_cfg)
                       if "://" in args.floor else args.floor)
        cmd += ["--floor", floor_local]
    print("[worker] training:", " ".join(cmd))
    rc = subprocess.run(cmd).returncode
    if rc != 0:
        print(f"[worker] gen0 harness failed (rc={rc})")
        return rc

    # PUSH the versioned artifact bundle (weights.json + manifest.json) to the bucket.
    bundle = os.path.join(out_dir, args.version)
    out_prefix = args.out_uri.rstrip("/")
    pushed = []
    for fn in sorted(os.listdir(bundle)):
        uri = f"{out_prefix}/weights/{args.version}/{fn}"
        storage.put(os.path.join(bundle, fn), uri, config=storage_cfg)
        pushed.append(uri)
    print(f"[worker] train done: pushed {len(pushed)} artifact(s):")
    for u in pushed:
        print("   ", u)
    return 0


def build_parser() -> argparse.ArgumentParser:
    p = argparse.ArgumentParser(prog="python -m backend.worker",
                                description="Storage-aware worker: pull ? run pipeline ? push.")
    sub = p.add_subparsers(dest="cmd", required=True)

    pi = sub.add_parser("infer", help="run inference on one video URI and push outputs")
    pi.add_argument("--video-uri", required=True, help="local path / local:// / s3:// / gcs://")
    pi.add_argument("--out-uri", required=True, help="output prefix (outputs/<video_id>/? is appended)")
    pi.add_argument("--segmenter", default=None, help="default: indexing.default_segmenter")
    pi.add_argument("--config", default=None, help="config.json path (default: ./config.json)")
    pi.add_argument("--scratch", default=None, help="local scratch dir (default: a temp dir)")
    pi.add_argument("--max-frames", type=int, default=None)
    pi.add_argument("--set", action="append", default=[], metavar="k=v",
                    help="config override, e.g. --set indexing.skip_ai_handoff=true")
    pi.set_defaults(func=cmd_infer)

    pt = sub.add_parser("train", help="Gen-0 train-from-bucket (labels [+videos] -> trajectories -> harness)")
    pt.add_argument("--corpus-uri", required=True, help="prefix containing labels/ (+ videos/ for "
                    "--trajectory-source detector), e.g. s3://bucket or gcs://bucket")
    pt.add_argument("--out-uri", required=True, help="output prefix (weights/<version>/ is appended)")
    pt.add_argument("--version", required=True, help="semver for the artifact bundle")
```

```
    pt.add_argument("--trajectory-source", choices=["synth", "detector"], default="synth",
                    help="synth = CPU-mock label-consistent trajectories (default, no GPU); "
                         "detector = REAL shuttle trajectories via the resolved DetectorRunner "
                         "(needs videos/ in the corpus; honours detector_impl).")
    pt.add_argument("--segmenter", default="wasb_hybrid",
                    help="trajectory hybrid whose detector is used for --trajectory-source detector")
    pt.add_argument("--floor", default=None, help="floor baseline JSON (local path or storage URI)")
    pt.add_argument("--config", default=None)
    pt.add_argument("--scratch", default=None)
    pt.add_argument("--fps", type=float, default=30.0, help="synth trajectory fps (detector probes the video)")
    pt.add_argument("--frame-width", type=float, default=1280.0,
                    help="synth trajectory frame width (detector probes the video)")
    pt.add_argument("--set", action="append", default=[], metavar="k=v")
    pt.set_defaults(func=cmd_train)
    return p


def main(argv=None) -> int:
    args = build_parser().parse_args(argv)
    return args.func(args)


if __name__ == "__main__":
    sys.exit(main())

```

16. assistant (2026-06-20T08:57:40.821Z)

Now let me verify the critical model_dump survival concern empirically. The key question: does `config.get("indexing", {})` produce a dict whose `wasb` sub-dict contains `device` and `python_bin`? Let me run a quick check.

17. user (2026-06-20T08:57:44.592Z)

```
type idx: <class 'dict'>
type wasb: <class 'dict'>
device in wasb: True -> cuda
python_bin in wasb: True -> python
resolved device: cuda python_bin: python weights: ~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar
```

18. assistant (2026-06-20T08:57:46.977Z)

The model_dump survival works. Now let me verify env override behavior, including the empty-env-var fall-through concern, and a non-default config value.

19. user (2026-06-20T08:57:52.583Z)

```
CONFIG-ONLY  device=mps python=/opt/conda/envs/wasb/bin/python weights=/cfg/weights.pth
ENV-WINS     device=cpu python=/env/python weights=/env/weights.pth
EMPTY-ENV    device=mps python=/opt/conda/envs/wasb/bin/python weights=/cfg/weights.pth
```

20. assistant (2026-06-20T08:57:57.347Z)

Env precedence and empty-env fall-through all work correctly (because `env(...) or w.get(...)` treats `""` as falsy). Now let me verify the scratch_dir env-only resolution and the `WASB_DEVICE` not being in `WasbIndexConfig` but documented. Let me check the `scratch_dir` precedence subtlety: `from_indexing_cfg` reads `scratch_dir` ONLY from env, never from `w.get("scratch_dir")`. But `WasbIndexConfig` has no `scratch_dir` field anyway. Let me check the resolution behavior of each family and the tracknet fall-through.

21. user (2026-06-20T08:58:08.429Z)

```
WASB -> NativeWasbRunner {'weights_path': '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar', 'detector_impl': 'native', 'device':
'cuda'}
FUSION wasb -> NativeWasbRunner {'weights_path': '~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar', 'detector_impl': 'native',
'fusion_substrate': 'wasb', 'device': 'cuda'}
FUSION tracknet -> NotImplementedError: detector_impl='native' supports the WASB substrate only
? se
TRACKNET -> NotImplementedError: detector_impl='native' is not supported for the 'TrackNet'
family ? only WASB ha
```

22. assistant (2026-06-20T08:58:12.531Z)

All families resolve correctly. Now let me check the alias set and the case-handling, plus a subtle concern: `_resolve_runner` lowercases the impl. Let me verify the alias set and also check a potential issue around `RALLY_DETECTOR_IMPL` empty-string handling and the `WASB_DEVICE` documentation gap. Let me also check `wasb_infer.py` for the `--device` threading and parity (default cuda keeping Hydra overrides identical).

23. user (2026-06-20T08:58:13.516Z)

```
"""WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).

WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
own building blocks (ImageDataset transform + detector + online tracker) to run on
any folder of extracted frames and emit a trajectory CSV in the exact shape the
indexer's `tracknet_runner.parse_trajectory_csv` already consumes:

    Frame,Visibility,X,Y

It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
(`wasb_runner.py`) stages this file + the frames into WSL and invokes it.

Resumable across reboots
------------------------
The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
the slow, expensive part; the CPU tracker pass that turns detections into a
trajectory is cheap and *stateful* (it must see every frame in order, so it can't
resume mid-stream). We exploit that split:

  * The detector pass writes its per-window raw outputs incrementally to
    ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
    records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
    most ``--flush-every`` windows of GPU work instead of the whole pass.
  * On restart we replay the cached windows, skip the DataLoader ahead to the first
    un-cached window, and continue. Once every window is cached, the detector pass
    is skipped entirely and we just re-run the (cheap, deterministic) tracker over
    the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
    not the GPU hour.

The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
weights, sport, window size, or frame count change.

Usage (in WSL, from ~/models/WASB-SBDT/src):
    python wasb_infer.py --frames_dir /path/frames --weights
../pretrained_weights/wasb_badminton_best.pth.tar \
        --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
```

```
every 1000] [--fresh]
"""
import os
import os.path as osp
import csv
import glob
import json
import argparse
import logging
from collections import defaultdict
from contextlib import contextmanager

logger = logging.getLogger(__name__)

CACHE_VERSION = 1
_DET_FILE = "det_raw.jsonl"
_MANIFEST_FILE = "manifest.json"


def _build_cfg(weights: str, sport: str, device: str = "cuda"):
    """Compose the WASB eval config via Hydra, overriding model/weights/device.

    ``device`` is ``cuda`` (default ? the historical WSL behaviour, byte-parity preserved),
    ``mps`` (Apple) or ``cpu``. Only the single-GPU CUDA path requests ``runner.gpus=[0]``;
    cpu/mps run with no GPU list so the detector places tensors on ``device``.
    """
    from hydra import compose, initialize
    gpus = "[0]" if device == "cuda" else "[]"   # single GPU on cuda; none for cpu/mps
    with initialize(config_path="configs", version_base=None):
        cfg = compose(config_name="eval", overrides=[
            f"dataset={sport}",
            "model=wasb",
            f"detector.model_path={weights}",
            f"runner.device={device}",
            f"runner.gpus={gpus}",  # default config assumes multiple GPUs; pin to this box
        ])
    return cfg


def _frame_id(path: str) -> int:
    """Best-effort integer frame id from a frame filename (e.g. frame_000180.png -> 180)."""
    stem = osp.splitext(osp.basename(path))[0]
    digits = "".join(ch for ch in stem if ch.isdigit())
    return int(digits) if digits else -1


# --------------------------------------------------------------------------- #
# Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
# --------------------------------------------------------------------------- #
def _cache_paths(cache_dir: str):
    return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)


def default_cache_dir(out_csv: str) -> str:
    """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
    d = osp.dirname(osp.abspath(out_csv))
    stem = osp.splitext(osp.basename(out_csv))[0]
    return osp.join(d, stem + "_wasbcache")


def _atomic_write_text(path: str, text: str) -> None:
```

```
    """Write then rename so a crash mid-write can't leave a half-written file."""
    tmp = path + ".tmp"
    with open(tmp, "w") as f:
        f.write(text)
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, path)


def _det_plain(det) -> list:
    """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able list."""
    xy = det["xy"]
    return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]


def _dets_to_tracker(plain_list):
    """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy array,
    the tracker does vector math / np.linalg.norm on it)."""
    import numpy as np
    return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
            for p in plain_list]


def write_manifest(cache_dir: str, manifest: dict) -> None:
    _, mpath = _cache_paths(cache_dir)
    _atomic_write_text(mpath, json.dumps(manifest, indent=2))


def read_manifest(cache_dir: str):
    _, mpath = _cache_paths(cache_dir)
    if not osp.exists(mpath):
        return None
    try:
        with open(mpath) as f:
            return json.load(f)
    except (json.JSONDecodeError, OSError):
        return None


def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
                        frames_in: int, frames_total: int, device: str = "cuda") -> bool:
    """A cache is reusable only if the run that produced it matches this run's inputs.

    ``device`` defaults to ``cuda`` so caches written before device-keying (no ``device``
    field) stay compatible with a CUDA run ? but a cpu/mps run will NOT reuse a cuda cache
    (detections can differ numerically across devices), and vice-versa.
    """
    if not manifest or manifest.get("version") != CACHE_VERSION:
        return False
    return (
        manifest.get("frames_dir") == osp.abspath(frames_dir)
        and manifest.get("weights") == weights
        and manifest.get("sport") == sport
        and manifest.get("frames_in") == frames_in
        and manifest.get("frames_total") == frames_total
        and manifest.get("device", "cuda") == device
    )


def reset_cache(cache_dir: str) -> None:
```

```
    """Drop any existing cache so the next run starts clean."""
    det_jsonl, mpath = _cache_paths(cache_dir)
    for p in (det_jsonl, mpath):
        if osp.exists(p):
            os.remove(p)


def load_and_clean_cache(cache_dir: str):
    """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
    rebuild the per-frame detection accumulation, and rewrite the file to exactly that
    prefix so a trailing half-written line from a crash can't corrupt future appends.

    Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
    [x, y, score, scale] candidates (accumulated across every window the frame is in,
    in window order ? identical to a non-resumed run).
    """
    det_jsonl, _ = _cache_paths(cache_dir)
    det_by_fid = defaultdict(list)
    valid: list = []
    if osp.exists(det_jsonl):
        with open(det_jsonl, "r") as f:
            for line in f:
                s = line.strip()
                if not s:
                    continue
                try:
                    obj = json.loads(s)
                except json.JSONDecodeError:
                    break                       # truncated trailing line (crash mid-flush)
                if obj.get("w") != len(valid):  # non-contiguous -> stop at the gap
                    break
                valid.append(s)
                for fid, plain in obj["f"]:
                    det_by_fid[int(fid)].extend([list(p) for p in plain])
        # Guarantee a clean append boundary by rewriting only the good prefix.
        _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
    return len(valid), det_by_fid


def _append_windows(fh, objs) -> None:
    """Append window records as JSONL and durably flush (bounded loss on crash)."""
    for o in objs:
        fh.write(json.dumps(o, separators=(",", ":")) + "\n")
    fh.flush()
    os.fsync(fh.fileno())


def _atomic_write_trajectory(out_csv: str, rows) -> None:
    os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
    tmp = out_csv + ".tmp"
    with open(tmp, "w", newline="") as f:
        w = csv.writer(f)
        w.writerow(["Frame", "Visibility", "X", "Y"])
        w.writerows(rows)
        f.flush()
        os.fsync(f.fileno())
    os.replace(tmp, out_csv)


# ------------------------------------------------------------------------ #
# Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
# ------------------------------------------------------------------------ #
_STREAM_FRAME_FMT = "{:08d}.png"

```

```

def synthetic_frame_path(index: int) -> str:
    """Stable, index-encoding frame name used by the streaming path.

    It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
    downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
    frames came from disk or from the in-memory stream. The streaming image loader
    inverts this name back to an index to fetch the decoded frame.
    """
    return _STREAM_FRAME_FMT.format(int(index))


def build_windows(frames: list, frames_in: int):
    """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
    + checkpoint). Pure list math, identical for the disk and streaming paths.

    Returns a list of windows, each a list of ``frames_in`` consecutive paths
    (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
    caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
    ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
    """
    return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]


class SequentialFrameStore:
    """In-memory sliding buffer over a forward-only frame reader, sized for the
    detector's sliding-window access pattern (peak O(window) frames, not O(video)).

    The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
    ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
    in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
    start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
    boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
    far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
    earliest index any not-yet-finished window can still ask for) and decode each source
    frame exactly once via the forward-only ``reader(index)``.
    """

    def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
        self._reader = reader
        self._total = int(total)
        self._window = max(1, int(window))
        self._buf: dict = {}
        # On a resumed run the detector's first needed frame is `start_index`; begin
        # pulling there so the reader can skip (seek past) the already-cached prefix
        # instead of decoding 0..start_index-1 just to throw them away.
        self._next = int(start_index)       # next index not yet pulled from the reader
        self._max_seen = int(start_index) - 1
        self._floor = int(start_index)      # lowest index we still keep (never evict below)

    def get(self, index: int):
        index = int(index)
        if index < self._floor:
            raise KeyError(
                f"frame {index} already evicted (below floor {self._floor}); the access "
                f"pattern must stay within a window of the highest frame seen")
        if index >= self._total:
            raise KeyError(f"frame {index} out of range (total {self._total})")
        # Pull forward from the reader until the requested index is buffered.
        while self._next <= index:
            self._buf[self._next] = self._reader(self._next)
            self._next += 1
        if index > self._max_seen:
```

```
            self._max_seen = index
        # Evict frames older than the earliest a still-running window could re-request:
        # once we've seen index `m`, no future window starts before `m - (window-1)`.
        new_floor = max(self._floor, self._max_seen - (self._window - 1))
        for k in range(self._floor, new_floor):
            self._buf.pop(k, None)
        self._floor = new_floor
        return self._buf[index]


def _index_from_synthetic_path(path: str) -> int:
    """Invert ``synthetic_frame_path``: a frame path -> its integer source index.

    Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
    in lockstep with the on-disk naming scheme.
    """
    return _frame_id(path)


def _bgr_to_pil_rgb(frame_bgr):
    """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the EXACT
    PIL RGB image WASB's disk path produces.

    The disk path is ``frame_bgr -> cv2.imwrite(PNG) -> Image.open(PNG).convert('RGB')``;
    because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr, COLOR_BGR2RGB)``
    bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the streamed
    frame indistinguishable from the on-disk one to everything downstream.
    """
    import cv2
    from PIL import Image
    return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))


def _make_streamed_read_image(store, real_read_image):
    """Build the ``read_image`` replacement used during a streaming detector pass.

    A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB image,
    matching ``read_image``'s real return type); any path that doesn't parse to an index falls
    through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-testable
    without the WSL-only ``dataloaders`` package.
    """
    def _read_image(path, *args, **kwargs):
        idx = _index_from_synthetic_path(path)
        if idx < 0:
            return real_read_image(path, *args, **kwargs)
        return _bgr_to_pil_rgb(store.get(idx))
    return _read_image


@contextmanager
def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index: int = 0):
    """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory decoded
    frames during the detector pass, byte-for-byte as it would over real PNGs.

    WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
    ``cv2.imread``. ``read_image`` (``utils.utils``) does
```

```
``Image.open(path).convert('RGB')``,
    returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore feed it a
    PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
    byte-identical to the disk round-trip, and the shim never touches the filesystem so the
    ``osp.exists`` guard is sidestepped for synthetic stream paths.

    We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer actually
    calls (``dataset_loader`` does ``from utils import read_image``, so the name lives in that
    module's namespace; patching ``utils.read_image`` would NOT rebind the already-imported
    reference).

    ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder wrapped
    in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``); on a
    resume we start at ``start_index`` so the decoder seeks past already-cached windows. The
    original reader is restored on exit.
    """
    from dataloaders import dataset_loader as _dl
    store = SequentialFrameStore(frame_loader, total, window=window, start_index=start_index)
    real_read_image = _dl.read_image
    # Rebind read_image in the consuming module for the duration of the detector pass;
    # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff B010-clean.)
    _dl.read_image = _make_streamed_read_image(store, real_read_image)  # type: ignore[assignment]
    try:
        yield store
    finally:
        _dl.read_image = real_read_image  # type: ignore[assignment]


# --------------------------------------------------------------------------- #
# Inference
# --------------------------------------------------------------------------- #
def run(frames, weights: str, out_csv: str, sport: str = "badminton",
        limit: int = 0, batch_size: int = 8, num_workers: int = 4,
        log_every_batches: int = 50, cache_dir: "str | None" = None,
        flush_every: int = 1000, fresh: bool = False,
        frame_loader=None, source_key: "str | None" = None,
        device: str = "cuda") -> str:
    """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.

    Two equivalent ways to supply pixels (output is bit-identical between them):

    * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
      globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via ``cv2.imread``.
    * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
      frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
      pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
      reader, which returns a PIL RGB image) over the DataLoader pass so every other line of
      ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is identical
      to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
      ``cvtColor(bgr, BGR2RGB)``, verified byte-identical). Streaming forces ``num_workers=0``
```

```
          (the in-process shim + buffer can't cross worker processes).

        ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
        frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
        after reboot still matches.
        """
        import time
        import torch
        from torch.utils.data import DataLoader
        from detectors import build_detector
        from trackers import build_tracker
        from dataloaders import build_img_transforms, build_seq_transforms
        from dataloaders.dataset_loader import ImageDataset
        from utils import Center

        streaming = frame_loader is not None

        cfg = _build_cfg(weights, sport, device)
        frames_in = int(cfg["model"]["frames_in"])
        input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
        output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))

        if streaming:
            # `frames` is already the ordered list of (synthetic) frame paths.
            if not isinstance(frames, (list, tuple)):
                raise SystemExit("streaming mode requires `frames` to be a list of frame paths")
            frames = list(frames)
            frames_dir = source_key or "stream"
        else:
            frames_dir = frames
            frames = sorted(glob.glob(osp.join(frames_dir, "*.png")) + glob.glob(osp.join(frames_dir, "*.jpg")))
        if limit:
            frames = frames[:limit]
        if len(frames) < frames_in:
            where = frames_dir if not streaming else "video stream"
            raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")

        # Sliding windows of `frames_in` consecutive frames (the unit of GPU work + checkpoint).
        samples = []
        for window in build_windows(frames, frames_in):
            annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)} for p in window]
            samples.append({"frames": window, "annos": annos})
        windows_total = len(samples)

        # ---- cache setup / resume decision ---------------------------------- #
        if cache_dir is None:
            cache_dir = default_cache_dir(out_csv)
        os.makedirs(cache_dir, exist_ok=True)
        det_jsonl, _ = _cache_paths(cache_dir)
        cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
        manifest = None if fresh else read_manifest(cache_dir)
        resume = manifest_compatible(
            manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
            frames_in=frames_in, frames_total=len(frames), device=device,
        )
        if resume:
            windows_done, det_by_fid = load_and_clean_cache(cache_dir)
            logger.info(f"resuming from cache: {windows_done}/{windows_total} windows already detected")
        else:
            if manifest is not None:
```

```
            logger.info("cache incompatible with this run -> starting fresh")
        reset_cache(cache_dir)
        windows_done, det_by_fid = 0, defaultdict(list)

    base_manifest = {
        "version": CACHE_VERSION,
        "frames_dir": cache_key,
        "weights": weights,
        "sport": sport,
        "frames_in": frames_in,
        "frames_total": len(frames),
        "device": device,
        "windows_total": windows_total,
        "windows_done": windows_done,
    }
    write_manifest(cache_dir, base_manifest)

    # ---- detector pass over the un-cached windows ----------------------- #
    remaining = samples[windows_done:]
    if remaining:
        _, transform_test = build_img_transforms(cfg)
        try:
            _, seq_transform_test = build_seq_transforms(cfg)
        except Exception:
            seq_transform_test = None
        ds = ImageDataset(cfg, remaining, input_wh, output_wh,
                          transform=transform_test, seq_transform=seq_transform_test, is_train=False)
        # Streaming forces single-process loading: the in-memory frame store + the
        # cv2.imread shim live in THIS process and can't be pickled to DataLoader workers.
        loader_workers = 0 if streaming else num_workers
        loader = DataLoader(ds, batch_size=batch_size, shuffle=False, num_workers=loader_workers)
        detector = build_detector(cfg)

        from contextlib import ExitStack

        t0 = time.time()
        done = windows_done
        win_idx = windows_done
        log_every = max(1, log_every_batches)
        flush_n = max(1, flush_every)
        pending = []
        logger.info(f"detecting on {len(remaining)} remaining windows "
                    f"(total {windows_total}, batch={batch_size}, workers={loader_workers}, "
                    f"source={'video-stream' if streaming else 'frames-dir'})...")
        det_f = open(det_jsonl, "a")
        try:
            with ExitStack() as stack:
                stack.enter_context(torch.no_grad())
                # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)` to the
                # in-memory decoded frame for that synthetic path; every other line of
                # __getitem__ runs unchanged so the tensor matches the PNG round-trip exactly.
                # The detector's first needed frame is `windows_done` (window i starts at
                # frame i), so prime the store there for a resume.
                if streaming:
                    stack.enter_context(
                        _streaming_read_image_patch(frame_loader, len(frames),
                                                    window=frames_in, start_index=windows_done))
```

```
                for bi, batch in enumerate(loader):
                    imgs, _hms, trans = batch[0], batch[1], batch[2]
                    img_paths = [list(t) for t in batch[-1]]   # [frame_in][batch] -> path
                    batch_results, _ = detector.run_tensor(imgs, trans)
                    nb = imgs.shape[0]
                    for ib in range(nb):
                        frames_payload = []
                        for ie in sorted(batch_results[ib].keys()):
                            p = img_paths[ie][ib]
                            fid = _frame_id(p)
                            plain = [_det_plain(d) for d in batch_results[ib][ie]]
                            frames_payload.append([fid, plain])
                            det_by_fid[fid].extend(plain)
                        pending.append({"w": win_idx, "f": frames_payload})
                        win_idx += 1
                    done += nb
                    if len(pending) >= flush_n or done >= windows_total:
                        _append_windows(det_f, pending)
                        pending = []
                        base_manifest["windows_done"] = done
                        write_manifest(cache_dir, base_manifest)
                    if (bi + 1) % log_every == 0 or done >= windows_total:
                        el = time.time() - t0
                        new_done = done - windows_done
                        rate = new_done / el if el > 0 else 0.0
                        eta = (windows_total - done) / rate if rate > 0 else 0.0
                        logger.info(f"{done}/{windows_total} "
                                    f"({100 * done // max(1, windows_total)}%) | {rate:.1f} win/s | "
                                    f"elapsed {el:.0f}s | ETA {eta:.0f}s")
                if pending:
                    _append_windows(det_f, pending)
                    base_manifest["windows_done"] = done
                    write_manifest(cache_dir, base_manifest)
        finally:
            det_f.close()
        logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
    else:
        logger.info("detector cache already complete -> tracker-only re-run")

    # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
    tracker = build_tracker(cfg)
    tracker.refresh()
    rows = []
    for fid in sorted(det_by_fid.keys()):
        r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
        rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))

    _atomic_write_trajectory(out_csv, rows)
    visible = sum(1 for _, v, *_ in rows if v)
    logger.info(f"wrote {len(rows)} rows -> {out_csv}")
    logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
    return out_csv


def extract_frames(video_path: str, frames_dir: str) -> str:
    """Decode a video to PNG frames named by source frame index (00000000.png ...).

    Idempotent: if the dir already holds the expected number of frames (matching the
    container's reported frame count) we skip re-decoding ? re-extraction after a
    reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
    extracting with ffmpeg on the host and passing --frames_dir.
    """
    import cv2
```

```
    os.makedirs(frames_dir, exist_ok=True)
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise SystemExit(f"cannot open video: {video_path}")
    expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    existing = len(glob.glob(osp.join(frames_dir, "*.png")))
    if expected > 0 and existing >= expected:
        cap.release()
        logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
        return frames_dir
    i = 0
    while True:
        ok, frame = cap.read()
        if not ok:
            break
        cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
        i += 1
    cap.release()
    logger.info(f"extracted {i} frames -> {frames_dir}")
    return frames_dir


def _probe_frame_count(video_path: str) -> int:
    """Container-reported frame count via cv2 (0 if unknown). Used to size the stream."""
    import cv2
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise SystemExit(f"cannot open video: {video_path}")
    n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    cap.release()
    return n


def make_video_frame_loader(video_path: str):
    """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.

    ``frame_loader(index) -> BGR uint8 ndarray`` decodes the video forward-only with one
    long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
    detector's sliding-window order); it reads sequentially and only uses a frame-seek to
    skip forward when a *resume* starts past the current position. Each ``cap.read()``
    returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG would
    yield (PNG is lossless), so the detector sees identical pixels to the disk path.

    Returns the count hint from the container so the caller can build the frame list;
    ``release`` closes the capture.
    """
    import cv2
    cap = cv2.VideoCapture(video_path)
    if not cap.isOpened():
        raise SystemExit(f"cannot open video: {video_path}")
    count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
    state = {"pos": 0}  # next index cap.read() will return

    def frame_loader(index: int):
        index = int(index)
        if index < state["pos"]:
            raise RuntimeError(
                f"streaming decode is forward-only; got index {index} after {state['pos']}")
        if index > state["pos"]:
            # Forward skip only happens when resuming past cached windows. Seek once.
            cap.set(cv2.CAP_PROP_POS_FRAMES, index)
```

```
            state["pos"] = index
        ok, frame = cap.read()
        if not ok:
            raise RuntimeError(f"failed to decode frame {index} from {video_path}")
        state["pos"] += 1
        return frame

    def release():
        cap.release()

    return frame_loader, count_hint, release


def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str = "badminton",
                        limit: int = 0, batch_size: int = 8,
                        log_every_batches: int = 50, cache_dir: "str | None" = None,
                        flush_every: int = 1000, fresh: bool = False,
                        device: str = "cuda") -> str:
    """Fast path: decode ``video_path`` on the fly and feed frames straight to the
    detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
    dominates a long clip). Output is bit-identical to the frames-on-disk path; see
    ``run``'s docstring for why.
    """
    frame_loader, count_hint, release = make_video_frame_loader(video_path)
    try:
        if count_hint <= 0:
            raise SystemExit(
                f"could not determine frame count for {video_path}; cannot stream "
                f"(fall back to frame extraction with --frames_dir)")
        frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
        source_key = "video:" + osp.abspath(video_path)
        return run(frame_list, weights, out_csv, sport, limit,
                   batch_size=batch_size, num_workers=0,
                   log_every_batches=log_every_batches, cache_dir=cache_dir,
                   flush_every=flush_every, fresh=fresh,
                   frame_loader=frame_loader, source_key=source_key, device=device)
    finally:
        release()


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
    ap.add_argument("--video", help="video file; frames are extracted internally (in WSL)")
    ap.add_argument("--frames_out_dir", help="where to extract frames when --video is used")
    ap.add_argument("--stream-video", action="store_true",
                    help="FAST PATH: decode --video on the fly and feed frames straight to "
                         "the detector (no PNG extraction). Output-equivalent to the disk path.")
    ap.add_argument("--weights", required=True)
    ap.add_argument("--out", required=True)
    ap.add_argument("--sport", default="badminton")
    ap.add_argument("--device", default="cuda", choices=["cuda", "mps", "cpu"],
                    help="torch device for the detector (default cuda = the WSL parity path)")
    ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick tests)")
    ap.add_argument("--batch-size", type=int, default=8)
    ap.add_argument("--num-workers", type=int, default=4)
    ap.add_argument("--log-every-batches", type=int, default=50, help="progress log cadence")
```

```
    ap.add_argument("--cache-dir", default=None,
                    help="resumable detector cache dir (default: <out>_wasbcache next to --out)")
    ap.add_argument("--flush-every", type=int, default=1000,
                    help="fsync the detector cache every N windows (bounds reboot loss)")
    ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and start over")
    args = ap.parse_args()

    if args.stream_video:
        if not args.video:
            raise SystemExit("--stream-video requires --video")
        run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
                            batch_size=args.batch_size,
                            log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
                            flush_every=args.flush_every, fresh=args.fresh, device=args.device)
        return

    frames_dir = args.frames_dir
    if args.video:
        frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video + "_frames"))
    if not frames_dir:
        raise SystemExit("provide --frames_dir or --video")
    run(frames_dir, args.weights, args.out, args.sport, args.limit,
        batch_size=args.batch_size, num_workers=args.num_workers,
        log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
        flush_every=args.flush_every, fresh=args.fresh, device=args.device)


if __name__ == "__main__":
    main()

```

24. assistant (2026-06-20T08:58:21.413Z)

Now I've confirmed the parity contract: default `device="cuda"` keeps the Hydra overrides identical (`runner.device=cuda`, `runner.gpus=[0]`), and the manifest's `device` defaults to `"cuda"` for compat.

Now let me dig into a subtle parity concern. The native runner passes `--device` always (line 174 of native_wasb_runner.py), even when `device="cuda"`. The WSL runner does NOT pass `--device` at all. The `wasb_infer.py` default is `cuda`, so passing `--device cuda` is equivalent to omitting it. Good ? parity holds.

But there's a more important parity concern: the WSL runner passes the CSV out path via `--out wsl_out_dir + '/' + key + '_wasb.csv'`, while the native runner passes `--out out_csv` (an abspath). Both reach the same `_atomic_write_trajectory`. Fine.

Now let me look at a real parity risk: the **cache-dir keying**. The native streaming path uses `source_key = "video:" + osp.abspath(video_path)`. On Linux native vs WSL, the abspath differs, but the cache is per-box so that doesn't affect byte-parity of the CSV. Let me now check the tests to understand what's covered and verify they pass.

25. user (2026-06-20T08:58:21.769Z)

```
"""NativeWasbRunner coverage ? the Linux-native (no-WSL) WASB detector path (M3.5).

The native subprocess is mocked (no torch/conda/GPU here). These tests lock the contract
the GPU-box byte-parity gate then relies on: it IS a DetectorRunner, it is NOT a WSL runner
(no WslCommandMixin / wsl invocation), it builds a native argv (no `wsl`, no `conda
```

```
activate`, no `/mnt` translation) that runs the SAME wasb_infer.py with the SAME output CSV
name as the WSL runner, threads the device through, and cleans the frame cache on success.
"""
import os
import types
from unittest.mock import patch

import pytest

from backend.pipeline.detectors.native_wasb_runner import NativeWasbRunner, NativeWasbConfig
from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin


def _proc(rc=0, stdout="", stderr=""):
    return types.SimpleNamespace(returncode=rc, stdout=stdout, stderr=stderr)


# --- identity / no-WSL guarantee --------------------------------------------------------
def test_is_a_detector_runner():
    assert isinstance(NativeWasbRunner(NativeWasbConfig()), DetectorRunner)


def test_is_not_a_wsl_runner():
    """The whole point of M3.5: the native runner must carry NO WSL plumbing."""
    r = NativeWasbRunner(NativeWasbConfig())
    assert not isinstance(r, WslCommandMixin)
    assert not hasattr(r, "_wsl_bash")


def test_reuses_model_agnostic_windowing():
    from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
    assert NativeWasbRunner.parse_trajectory_csv is TrackNetRunner.parse_trajectory_csv
    assert NativeWasbRunner.trajectory_to_action_windows is TrackNetRunner.trajectory_to_action_windows


# --- config resolution (env overrides win over config over default) ---------------------
def test_from_indexing_cfg_defaults():
    cfg = NativeWasbConfig.from_indexing_cfg({})
    assert cfg.device == "cuda" and cfg.python_bin == "python"
    assert cfg.weights_path == "" and cfg.repo_dir == "~/models/WASB-SBDT"
    assert cfg.stream_video is False and cfg.keep_frames is False


def test_from_indexing_cfg_reads_config_block():
    cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
        "weights_path": "/m/w.pth.tar", "device": "cpu", "python_bin": "/envs/wasb/bin/python",
        "repo_dir": "/m/WASB-SBDT", "stream_video": True, "keep_frames": True, "sport": "tennis"}})
    assert cfg.weights_path == "/m/w.pth.tar" and cfg.device == "cpu"
    assert cfg.python_bin == "/envs/wasb/bin/python" and cfg.repo_dir == "/m/WASB-SBDT"
    assert cfg.stream_video is True and cfg.keep_frames is True and cfg.sport == "tennis"


def test_env_overrides_beat_config(monkeypatch):
    monkeypatch.setenv("WASB_WEIGHTS", "/env/w.pth.tar")
    monkeypatch.setenv("WASB_DEVICE", "mps")
    monkeypatch.setenv("WASB_PYTHON", "/env/py")
```

```
    monkeypatch.setenv("WASB_REPO_DIR", "/env/repo")
    monkeypatch.setenv("WASB_SCRATCH", "/env/scratch")
    cfg = NativeWasbConfig.from_indexing_cfg({"wasb": {
        "weights_path": "/cfg/w", "device": "cpu", "python_bin": "/cfg/py", "repo_dir": "/cfg/repo"}})
    assert cfg.weights_path == "/env/w.pth.tar" and cfg.device == "mps"
    assert cfg.python_bin == "/env/py" and cfg.repo_dir == "/env/repo"
    assert cfg.scratch_dir == "/env/scratch"


# --- healthcheck -----------------------------------------------------------------------
def _ok_cfg(tmp_path, **kw):
    w = tmp_path / "w.pth.tar"
    w.write_text("weights")
    (tmp_path / "repo" / "src").mkdir(parents=True)
    return NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "repo"), **kw)


def test_healthcheck_ok(tmp_path):
    r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
    with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda True")):
        ok, msg = r.healthcheck()
    assert ok and "torch" in msg


def test_healthcheck_empty_weights():
    ok, msg = NativeWasbRunner(NativeWasbConfig(weights_path="")).healthcheck()
    assert not ok and "weights" in msg.lower()


def test_healthcheck_weights_missing(tmp_path):
    (tmp_path / "repo" / "src").mkdir(parents=True)
    cfg = NativeWasbConfig(weights_path=str(tmp_path / "nope.pth"), repo_dir=str(tmp_path / "repo"))
    ok, msg = NativeWasbRunner(cfg).healthcheck()
    assert not ok and "not found" in msg.lower()


def test_healthcheck_repo_missing(tmp_path):
    w = tmp_path / "w.pth.tar"
    w.write_text("x")
    cfg = NativeWasbConfig(weights_path=str(w), repo_dir=str(tmp_path / "absent"))
    ok, msg = NativeWasbRunner(cfg).healthcheck()
    assert not ok and "repo" in msg.lower()


def test_healthcheck_torch_import_fails(tmp_path):
    r = NativeWasbRunner(_ok_cfg(tmp_path))
    with patch.object(r, "_run", return_value=_proc(1, stderr="ModuleNotFoundError: torch")):
        ok, msg = r.healthcheck()
    assert not ok and "failed" in msg.lower()


def test_healthcheck_cuda_unavailable_when_device_cuda(tmp_path):
    r = NativeWasbRunner(_ok_cfg(tmp_path, device="cuda"))
    with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")):
        ok, msg = r.healthcheck()
    assert not ok and "cuda" in msg.lower()


def test_healthcheck_cpu_device_ok_without_cuda(tmp_path):
```

```
    r = NativeWasbRunner(_ok_cfg(tmp_path, device="cpu"))
    with patch.object(r, "_run", return_value=_proc(0, stdout="torch 1.11.0 cuda False")):
        ok, msg = r.healthcheck()
    assert ok  # cpu run does not require a GPU


def test_healthcheck_python_not_found(tmp_path):
    r = NativeWasbRunner(_ok_cfg(tmp_path, python_bin="/no/such/python"))
    with patch.object(r, "_run", side_effect=FileNotFoundError()):
        ok, msg = r.healthcheck()
    assert not ok and "python" in msg.lower()


# --- run_predict ------------------------------------------------------------------------
def _drive_success(cfg, tmp_path, vid="abc123abc123", **patches):
    """Drive run_predict to success with the subprocess mocked; returns (runner, out, rm_spy, argv, cwd)."""
    r = NativeWasbRunner(cfg)
    key = f"clip__{vid[:12]}"
    (tmp_path / f"{key}_wasb.csv").write_text("Frame,Visibility,X,Y\n")  # success = CSV exists
    with patch.object(r, "_copy_infer_script", return_value=True), \
         patch.object(r, "_run", return_value=_proc(0)) as run_spy, \
         patch.object(r, "_rm_rf") as rm_spy:
        out = r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id=vid)
    argv = run_spy.call_args[0][0]
    cwd = run_spy.call_args.kwargs.get("cwd")
    return r, out, rm_spy, argv, cwd, key


def test_run_predict_builds_native_argv_no_wsl(tmp_path):
    _, out, rm, argv, cwd, key = _drive_success(NativeWasbConfig(weights_path="/m/w.pth", device="cuda"), tmp_path)
    assert out == os.path.join(os.path.abspath(str(tmp_path)), f"{key}_wasb.csv")
    # No WSL / conda plumbing anywhere in the command.
    assert "wsl" not in argv
    assert not any("conda" in a or "/mnt/" in a for a in argv)
    # It runs the SAME inference core, device-threaded, writing the SAME CSV name as WSL.
    assert argv[0] == "python" and argv[1] == "wasb_infer.py"
    assert "--device" in argv and argv[argv.index("--device") + 1] == "cuda"
    assert "--weights" in argv and argv[argv.index("--weights") + 1] == "/m/w.pth"
    assert argv[argv.index("--out") + 1] == out
    # Disk mode extracts frames + cleans them on success.
    assert "--frames_out_dir" in argv and "--stream-video" not in argv
    assert cwd.endswith(os.path.join("src"))
    rm.assert_called_once()


def test_run_predict_stream_mode_no_frames_dir(tmp_path):
    _, out, rm, argv, _cwd, _key = _drive_success(
        NativeWasbConfig(weights_path="/m/w.pth", stream_video=True), tmp_path)
    assert out is not None
    assert "--stream-video" in argv and "--frames_out_dir" not in argv
    rm.assert_not_called()  # streaming never materializes a frame cache


def test_run_predict_device_threaded_cpu(tmp_path):
    _, _out, _rm, argv, _cwd, _key = _drive_success(
        NativeWasbConfig(weights_path="/m/w.pth", device="cpu"), tmp_path)
    assert argv[argv.index("--device") + 1] == "cpu"


```

```python
def test_run_predict_keep_frames_skips_cleanup(tmp_path):
    _, _out, rm, _argv, _cwd, _key = _drive_success(
        NativeWasbConfig(weights_path="/m/w.pth", keep_frames=True), tmp_path)
    rm.assert_not_called()


def test_run_predict_none_when_copy_infer_fails(tmp_path):
    r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
    with patch.object(r, "_copy_infer_script", return_value=False):
        assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123") is None


def test_run_predict_none_when_subprocess_fails(tmp_path):
    r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
    with patch.object(r, "_copy_infer_script", return_value=True), \
         patch.object(r, "_run", return_value=_proc(1, stderr="CUDA error")):
        assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123") is None


def test_run_predict_none_when_csv_absent(tmp_path):
    # Subprocess "succeeds" (rc 0) but writes no CSV -> treated as failure.
    r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
    with patch.object(r, "_copy_infer_script", return_value=True), \
         patch.object(r, "_run", return_value=_proc(0)):
        assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123") is None


def test_run_predict_times_out_returns_none(tmp_path):
    import subprocess
    r = NativeWasbRunner(NativeWasbConfig(weights_path="/m/w.pth"))
    with patch.object(r, "_copy_infer_script", return_value=True), \
         patch.object(r, "_run", side_effect=subprocess.TimeoutExpired(cmd="x", timeout=1)):
        assert r.run_predict(r"C:\v\clip.mp4", str(tmp_path), video_id="abc123abc123") is None


def test_csv_name_matches_wsl_contract(tmp_path):
    """The native CSV name must equal the WSL runner's so downstream + the parity diff line up."""
    _, out, _rm, _argv, _cwd, key = _drive_success(NativeWasbConfig(weights_path="/m/w.pth"), tmp_path)
    assert os.path.basename(out) == f"{key}_wasb.csv"


# --- the detector_impl='native' seam (parallels the stub seam) -------------------------
def test_resolve_runner_native_selects_native_runner_for_wasb(monkeypatch):
    monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
    from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
    runner, params = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
    assert isinstance(runner, NativeWasbRunner)
    assert params.get("detector_impl") == "native"


def test_resolve_runner_native_env_overrides_config(monkeypatch):
    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
    from backend.pipeline.segmenters.wasb_hybrid import WasbHybridSegmenter
    runner, _ = WasbHybridSegmenter(None, {})._resolve_runner({"detector_impl": "auto"})
    assert isinstance(runner, NativeWasbRunner)

```

```

def test_resolve_runner_native_supported_for_fusion_wasb(monkeypatch):
    monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
    runner, params = FusionSegmenter(None, {})._resolve_runner({"detector_impl": "native"})
    assert isinstance(runner, NativeWasbRunner)
    assert params.get("fusion_substrate") == "wasb"


def test_resolve_runner_native_refused_for_fusion_tracknet(monkeypatch):
    monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
    from backend.pipeline.segmenters.fusion_hybrid import FusionSegmenter
    with pytest.raises(NotImplementedError):
        FusionSegmenter(None, {})._resolve_runner(
            {"detector_impl": "native", "fusion": {"substrate": "tracknet"}})


def test_resolve_runner_native_refused_for_tracknet(monkeypatch):
    monkeypatch.delenv("RALLY_DETECTOR_IMPL", raising=False)
    from backend.pipeline.segmenters.tracknet_hybrid import TrackNetHybridSegmenter
    with pytest.raises(NotImplementedError):
        TrackNetHybridSegmenter(None, {})._resolve_runner({"detector_impl": "native"})

```

26. assistant (2026-06-20T08:58:32.407Z)

Now I notice something important. There's a parity gap I should verify. The WSL `run_predict` (line 132) computes the key as `f"{stem}__{video_id[:12]}" if video_id else stem`, while the native (line 154) computes `f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")`. Same when video_id present. Good.

Now the **most important parity concern**: the WSL path passes `--out` pointing at the WSL-mounted dir which resolves to the Windows `output_win_dir`. But there's a subtle difference: the WSL runner does NOT pass a `--cache-dir`, so wasb_infer derives `default_cache_dir(out_csv)` = `<out>_wasbcache`. The native runner also doesn't pass `--cache-dir`. Both derive the cache next to the out CSV. That's consistent.

Now let me check a potential cross-platform issue in the native runner concerning `os.path.join` for the frames_dir and out paths when run on Linux. On Linux, `os.path.join` uses `/`. Fine. But the tests run on Windows. The test `test_run_predict_builds_native_argv_no_wsl` checks `cwd.endswith(os.path.join("src"))` and `argv[...] == out` where `out` is built with `os.path.join`. These are platform-consistent within a single run, so fine.

Let me now check a real concern: **does the WSL `WasbConfig.from_indexing_cfg` read env vars?** It does NOT. So if someone sets `WASB_WEIGHTS` env on a WSL box, the WSL runner ignores it but the native runner honors it. That's a documented asymmetry (native-only env). Now let me verify the test suite and worker tests pass, and run ruff/mypy.

27. user (2026-06-20T08:58:33.272Z)

```
"""Worker dispatcher: pull (storage) ? run pipeline ? push (storage).

`cmd_infer` is exercised end-to-end over the LOCAL storage backend with the validator +
segmenter mocked, so it needs no real video/GPU/network. The real S3 round-trip is validated
on the droplet against MinIO.
"""
import json

import pytest

from backend.config import load_config
from backend.worker import __main__ as wmain

```

```python

# ------------------------------------------------------------------- pure helpers

@pytest.mark.parametrize("raw,expected", [
    ("true", True), ("False", False), ("3", 3), ("2.5", 2.5), ("wasb_hybrid", "wasb_hybrid"),
])
def test_coerce(raw, expected):
    assert wmain._coerce(raw) == expected and type(wmain._coerce(raw)) is type(expected)


def test_apply_overrides_on_typed_config():
    cfg = _fresh_config()
    wmain._apply_overrides(cfg, ["indexing.skip_ai_handoff=true", "indexing.min_segment_duration=1.5",
                                 "sport=tennis"])
    assert cfg.indexing.skip_ai_handoff is True
    assert cfg.indexing.min_segment_duration == 1.5
    assert cfg.sport == "tennis"


def test_apply_overrides_rejects_bad_format():
    cfg = _fresh_config()
    with pytest.raises(ValueError):
        wmain._apply_overrides(cfg, ["no_equals_sign"])


def test_write_intervals_csv(tmp_path):
    segs = [{"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence": "LocalNoAI"},
            {"start_time": 9.0, "end_time": 12.5}]
    out = tmp_path / "intervals.csv"
    wmain._write_intervals_csv(segs, str(out))
    lines = out.read_text().splitlines()
    assert lines[0] == "start,end,shot_count,ending_reason"
    assert lines[1].startswith("2.000,5.000,4,")
    assert lines[2].startswith("9.000,12.500,,")


def test_write_report_json(tmp_path):
    out = tmp_path / "report.json"
    wmain._write_report_json("vid1", [{"start_time": 1.0, "end_time": 2.0}], "success", str(out))
    data = json.loads(out.read_text())
    assert data["video_id"] == "vid1" and data["segment_count"] == 1
    assert data["segments"][0] == {"start": 1.0, "end": 2.0}


def test_parser_infer_accepts_set_and_uris():
    args = wmain.build_parser().parse_args([
        "infer", "--video-uri", "s3://b/v.mp4", "--out-uri", "s3://b",
        "--set", "indexing.skip_ai_handoff=true", "--set", "sport=tennis"])
    assert args.cmd == "infer" and args.video_uri == "s3://b/v.mp4"
    assert args.set == ["indexing.skip_ai_handoff=true", "sport=tennis"]


# ------------------------------------------------------------------- cmd_infer (local, mocked)

class _OKResult:
    passed = True
    validator_name = "fake"
    message = "ok"

    def model_dump(self):
```

```
        return {"validator_name": "fake", "passed": True, "message": "ok"}


class _FailResult(_OKResult):
    passed = False
    message = "too blurry"


class _FakeSeg:
    def __init__(self, db, config):
        pass

    def process_video(self, video_id, path, max_frames=None):
        return [
            {"start_time": 2.0, "end_time": 5.0, "shot_count": 4, "shot_count_confidence": "LocalNoAI"},
            {"start_time": 9.0, "end_time": 12.0, "shot_count": 6, "shot_count_confidence": "LocalNoAI"},
        ]


def _fresh_config():
    # Load defaults without touching any cwd config.json.
    import tempfile
    p = tempfile.NamedTemporaryFile("w", suffix=".json", delete=False)
    p.write("{}")
    p.close()
    return load_config(p.name)


@pytest.fixture
def mocked_pipeline(monkeypatch):
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_OKResult()], {}))
    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)


def test_cmd_infer_local_roundtrip(tmp_path, mocked_pipeline):
    video = tmp_path / "in.mp4"
    video.write_bytes(b"FAKEVIDEO")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    out = tmp_path / "bucket"
    scratch = tmp_path / "scratch"

    rc = wmain.main([
        "infer", "--video-uri", str(video), "--out-uri", str(out),
        "--config", str(cfg), "--scratch", str(scratch),
        "--segmenter", "wasb_hybrid", "--set", "indexing.skip_ai_handoff=true",
    ])
    assert rc == 0
    intervals = out / "outputs" / "vid123" / "intervals.csv"
    report = out / "outputs" / "vid123" / "report.json"
    assert intervals.exists() and report.exists()
    rows = intervals.read_text().splitlines()
    assert len(rows) == 3                          # header + 2 rallies
    assert json.loads(report.read_text())["segment_count"] == 2


def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context",
                        lambda path, cfg: ([_FailResult()], {}))
```

```
    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
    video = tmp_path / "in.mp4"
    video.write_bytes(b"X")
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")
    rc = wmain.main(["infer", "--video-uri", str(video), "--out-uri", str(tmp_path / "o"),
                     "--config", str(cfg), "--scratch", str(tmp_path / "s")])
    assert rc == 2


# --- cmd_train: pull labels -> synth trajectories -> manifest -> harness -> push bundle ---

def test_cmd_train_orchestration(tmp_path, monkeypatch):
    import os
    import subprocess

    labels_dir = tmp_path / "corpus" / "labels"
    labels_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n2,9.0,12.0,let,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    # Mock the harness subprocess: write a fake versioned bundle at --out/--version.
    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(tmp_path / "corpus"), "--out-uri", str(out_dir),
                     "--version", "0.0.1", "--config", str(cfg), "--scratch", str(scratch)])
    assert rc == 0
    # bundle pushed to <out-uri>/weights/<version>/
    assert (out_dir / "weights" / "0.0.1" / "weights.json").exists()
    assert (out_dir / "weights" / "0.0.1" / "manifest.json").exists()
    # the manifest fed to the harness enumerated both labeled videos with a trajectory each
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2
    assert all("trajectory" in s and "labels" in s and "fps" in s for s in specs)


def test_parser_train_detector_args():
    args = wmain.build_parser().parse_args([
        "train", "--corpus-uri", "gcs://b", "--out-uri", "gcs://b", "--version", "1.0.0",
        "--trajectory-source", "detector", "--segmenter", "fusion_hybrid"])
    assert args.trajectory_source == "detector" and args.segmenter == "fusion_hybrid"


def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
```

```
    """--trajectory-source detector pulls videos + runs the resolved DetectorRunner. With
    RALLY_DETECTOR_IMPL=stub the *wiring* is exercised GPU-free (the stub IS the runner)."""
    import os
    import subprocess

    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # resolve a CPU stub runner (no GPU/WSL)
    corpus = tmp_path / "corpus"
    labels_dir = corpus / "labels"
    videos_dir = corpus / "videos"
    labels_dir.mkdir(parents=True)
    videos_dir.mkdir(parents=True)
    header = "rally_number,start_time,end_time,ending_reason,sport,shots_count\n"
    (labels_dir / "v1.rallies.csv").write_text(header + "1,2.0,5.0,winner,badminton,\n")
    (labels_dir / "v2.rallies.csv").write_text(header + "1,3.0,6.0,winner,badminton,\n")
    (videos_dir / "v1.mp4").write_bytes(b"FAKEVIDEO1")
    (videos_dir / "v2.mp4").write_bytes(b"FAKEVIDEO2")
    out_dir = tmp_path / "out"
    scratch = tmp_path / "scratch"
    cfg = tmp_path / "config.json"
    cfg.write_text("{}")

    def fake_run(cmd, *a, **k):
        out = cmd[cmd.index("--out") + 1]
        ver = cmd[cmd.index("--version") + 1]
        bdir = os.path.join(out, ver)
        os.makedirs(bdir, exist_ok=True)
        for fn in ("weights.json", "manifest.json"):
            with open(os.path.join(bdir, fn), "w") as f:
                f.write("{}")
        return subprocess.CompletedProcess(cmd, 0)
    monkeypatch.setattr(subprocess, "run", fake_run)

    rc = wmain.main(["train", "--corpus-uri", str(corpus), "--out-uri", str(out_dir),
                     "--version", "0.0.2", "--config", str(cfg), "--scratch", str(scratch),
                     "--trajectory-source", "detector", "--segmenter", "wasb_hybrid"])
    assert rc == 0
    specs = json.loads((scratch / "manifest.json").read_text())
    assert len(specs) == 2
    # Real-extract path: each spec points at an on-disk trajectory the runner actually wrote.
    assert all(os.path.exists(s["trajectory"]) for s in specs)
    assert all(s["trajectory"].endswith("_stub.csv") for s in specs)  # the stub runner's output

```

28. assistant (2026-06-20T08:58:35.929Z)

Now let me run the full focused test suite for these new tests plus run ruff/mypy on the changed files.

29. user (2026-06-20T08:58:46.122Z)

```
...........................................                              [100%]
43 passed in 6.52s
```

30. user (2026-06-20T08:58:49.797Z)

All checks passed!

31. user (2026-06-20T08:58:55.544Z)

backend\sports__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\pipeline\validators\base.py:36: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
backend\providers__init__.py:7: note: By default the bodies of untyped functions are not checked, consider using --check-untyped-defs [annotation-unchecked]
Success: no issues found in 4 source files

32. assistant (2026-06-20T08:59:04.403Z)

Clean. Now let me probe some adversarial edge cases I'm specifically tasked to find.

First: the `WASB_DEVICE` env var is read by `NativeWasbConfig.from_indexing_cfg` but NOT documented in the docstring's env list (lines 15-17 list only `WASB_WEIGHTS/WASB_REPO_DIR/WASB_PYTHON/WASB_SCRATCH`). The `WasbIndexConfig` comment (line 71) says "Env vars override these (WASB_DEVICE / WASB_PYTHON)". Minor doc inconsistency. Let me check a real seam concern: what happens when `_resolve_runner` is given a config where `idx_cfg` is a Pydantic model section vs dict. The worker calls `seg._resolve_runner(config.get("indexing", {}))` which returns a dict. The segmenter's `process_video` does `self.config.get("indexing", {})`. But in tests, `WasbHybridSegmenter(None, {})` passes `config={}` (a plain dict). Let me check whether `_build_native_runner` reads `idx_cfg.get("fusion", ...)` safely when idx_cfg is `{}`.

Let me check a subtle concern: when `detector_impl` is set via `--set indexing.detector_impl=native` in the worker, does it flow correctly? And does an uppercase/whitespace env value work? Let me test edge cases.

33. user (2026-06-20T08:59:15.591Z)

UPPERCASE env NATIVE -> NativeWasbRunner
WHITESPACE env -> resolved: WasbRunner
EMPTY env, config=native -> NativeWasbRunner

34. assistant (2026-06-20T08:59:24.005Z)

Found a behavioral edge: a whitespace-padded `RALLY_DETECTOR_IMPL=' native '` silently falls through to the WSL runner (not `.strip()`-ed). This is a minor footgun consistent with the pre-existing `stub` handling (the env var was never stripped before either), so it's pre-existing behavior, not a regression of this PR. I'll note it as a nit.

Now let me check a more serious concern: the worker `_resolve_train_detector` builds the segmenter with `seg_cls(None, config)`. For `FusionSegmenter`, `__init__` calls `super().__init__(db, config)`. Let me verify `BasePlaySegmenter.__init__` handles `db=None` and that `process_video` isn't called (only `_resolve_runner`). Also the `WasbHybridSegmenter(None, {})` in tests passes a dict config ? let me confirm `BasePlaySegmenter` doesn't choke.

35. user (2026-06-20T08:59:25.276Z)

```
from abc import ABC, abstractmethod
from typing import List, Dict, Any, Type, Optional
from backend.infrastructure.database import Database
from backend.results import Result  # new standardized contract (Result[Value] == Success[Value] | Failure)

class BasePlaySegmenter(ABC):
    def __init__(self, db: Database, config: Dict[str, Any]):
        self.db = db
        self.config = config

    @property
    @abstractmethod
    def name(self) -> str:
        """Unique identifier for this segmenter."""
        pass
```

```
    @property
    @abstractmethod
    def description(self) -> str:
        """Human-readable explanation of this segmenter."""
        pass

    @abstractmethod
    def process_video(self, video_id: str, video_path: str, player_pool: List[str] = None, max_frames: int = None) -> "Result[List[Dict[str, Any]]]":
        """
        Processes a video to detect play segments, populates SQLite database,
        and returns a Result.

        Use isinstance(result, Failure) (or result.is_success()) to distinguish
        "no segments found" from "something went wrong" (missing key, healthcheck
        failure, etc.). This is the start of the standardized error/result contract.
        """
        pass

class SegmenterRegistry:
    def __init__(self):
        self._segmenters: Dict[str, Type[BasePlaySegmenter]] = {}

    def register(self, segmenter_cls: Type[BasePlaySegmenter]):
        """Registers a new segmenter class under its name property."""
        try:
            temp_inst = segmenter_cls(None, {})
            name = temp_inst.name
        except Exception as e:
            raise ValueError(f"Failed to resolve name property from segmenter class {segmenter_cls.__name__}: {e}") from e
        self._segmenters[name] = segmenter_cls

    def get(self, name: str) -> Optional[Type[BasePlaySegmenter]]:
        return self._segmenters.get(name)

    def list_available(self) -> Dict[str, str]:
        """Returns a dict of registered segmenter names and their descriptions."""
        res = {}
        for name, cls in self._segmenters.items():
            try:
                temp_inst = cls(None, {})
                desc = temp_inst.description
            except Exception:
                desc = "No description available."
            res[name] = desc
        return res

segmenter_registry = SegmenterRegistry()
```

36. assistant (2026-06-20T08:59:32.756Z)

`db=None` is fine (just stored). Now the worker's `_resolve_train_detector` passes `config.get("indexing", {})` (a dict). For the fusion case, `_build_native_runner` reads `idx_cfg.get("fusion", {}).get("substrate", "wasb")` ? but in the worker path, `config.get("indexing", {})` returns the dumped dict which DOES include `fusion`. Let me verify that the substrate flows through model_dump too, and confirm the worker train path with detector_impl=native + fusion tracknet would raise cleanly (caught? `_resolve_train_detector` doesn't wrap `_resolve_runner` in try/except).

This is a real concern: in `_resolve_train_detector`, line 183 calls `seg._resolve_runner(...)` with NO try/except. If `detector_impl=native` + `fusion.substrate=tracknet` (or segmenter=tracknet_hybrid), `_resolve_runner` raises `NotImplementedError`, which propagates uncaught out of `cmd_train` ? crashes the worker with a traceback instead of the clean `return 2` + printed error pattern used everywhere else in the function. Let me confirm.

37. user (2026-06-20T08:59:41.706Z)

(Bash completed with no output)

38. assistant (2026-06-20T08:59:44.091Z)

Let me re-run capturing all output to see what happened.

39. user (2026-06-20T08:59:53.391Z)

RESULT: UNCAUGHT NotImplementedError -> detector_impl='native' is not supported for the 'TrackNet' family ? only WASB has a Linux-

40. assistant (2026-06-20T09:00:00.534Z)

Confirmed. `_resolve_train_detector` lets `NotImplementedError` from `_resolve_runner` propagate uncaught, crashing `cmd_train` with a traceback instead of the function's established "print error + return None ? return 2" pattern. This happens for a realistic misconfig: `--trajectory-source detector --segmenter tracknet_hybrid` (or `fusion_hybrid` with `fusion.substrate=tracknet`) while `detector_impl=native`. Same applies to `cmd_infer` ? `process_video` ? `_resolve_runner`, but there it's at least inside the segmenter call. Let me check whether `cmd_infer`'s `process_video` is inside a try. Looking at worker lines 115-141: yes, there's a `try/finally` but the `except` is not present ? only `finally` for the report. So a raise in `process_video` would propagate too, but the `finally` runs. Still an uncaught crash. However, `_resolve_train_detector` is the cleaner-to-fix and clearly-intended-to-be-graceful site (it already returns None on every other failure).

Let me verify the `cmd_infer` path too for completeness.

41. user (2026-06-20T09:00:09.684Z)

False <- cmd_infer catches NIE?
False <- cmd_infer has any except?

42. assistant (2026-06-20T09:00:21.128Z)

Now let me check one more subtle seam concern: the `from_indexing_cfg` reads `idx_cfg.get("wasb", {})`. But when called from the worker train path, `_build_native_runner(idx_cfg)` is called with the dict from `config.get("indexing", {})`. Wait ? actually `_resolve_runner` is what's called, and it calls `_build_native_runner(idx_cfg)`. Let me verify the `idx_cfg` passed there contains the `wasb` block (model_dump). Already confirmed it does.

Now a important question: does `WASB_DEVICE` env get picked up but `device` NOT being a documented env in the module docstring cause a mismatch with what the healthcheck/parity expects? No ? it's purely a doc completeness nit.

Let me check one genuinely important parity item: the native runner ALWAYS passes `--device` (even cuda), while the WSL runner NEVER passes `--device`. The acceptance gate says "device default 'cuda' must keep wasb_infer's Hydra overrides identical to before (parity)." Since `wasb_infer.py`'s `--device` defaults to `cuda` and `_build_cfg` produces `runner.device=cuda runner.gpus=[0]` for cuda ? which is exactly what the no-`--device` invocation produces ? the Hydra overrides ARE identical. Parity holds. Good.

Let me also confirm there's no issue with the native runner's `cwd=self._repo_src()` while the argv references `wasb_infer.py` (relative) ? matching the WSL runner's `cd .../src && python wasb_infer.py`. Consistent.

Now let me check whether `detector_impl` in `runner_params` could collide/overwrite anything in `detection_params` persisted. In `trajectory_hybrid.process_video`, `detection_params = {..., **runner_params}`. `runner_params` for native includes `detector_impl`, `device`, `weights_path`, `fusion_substrate`. None collide with the base keys. Fine.

Let me run the broader detector + segmenter test modules to make sure nothing regressed.

43. user (2026-06-20T09:00:38.098Z)

```
........................................................................ [ 27%]
........................................................................ [ 55%]
........................................................................ [ 82%]
.............................................                            [100%]
261 passed, 730 deselected in 13.84s
```

44. assistant (2026-06-20T09:00:40.623Z)

All green. Let me do a final check on the full suite quickly to be sure nothing else broke, since I'm asked to verify end-to-end.

45. user (2026-06-20T09:02:02.019Z)

```
........................................................................ [ 29%]
........................................................................ [ 36%]
........................................................................ [ 43%]
........................................................................ [ 50%]
........................................................................ [ 58%]
........................................................................ [ 65%]
........................................................................ [ 72%]
........................................................................ [ 79%]
........................................................................ [ 87%]
........................................................................ [ 94%]
.......................................................                  [100%]
991 passed in 77.46s (0:01:17)
========================================================================
 RESULT: PASS - all tests green
========================================================================
```

46. assistant (2026-06-20T09:02:06.585Z)

Full suite green (991 passed), ruff clean, mypy clean.

Let me do one last verification on the docstring env-list nit and the `WASB_SCRATCH`/`RALLY_SCRATCH`/`TMPDIR` precedence note, plus confirm the `device` field in `WasbIndexConfig` actually round-trips when set via `--set indexing.wasb.device=cpu` in the worker (the typed-config setattr path).

47. user (2026-06-20T09:02:15.885Z)

```
detector_impl: native
device: cpu python_bin: /x/py
```

48. assistant (2026-06-20T09:02:24.574Z)

The full `--set` override path works end-to-end. The seam wiring is sound. Let me compile my findings.

Summary of verification performed:
- `model_dump` survival of `WasbIndexConfig.device`/`python_bin` into `NativeWasbConfig.from_indexing_cfg`: VERIFIED works (nested section dumped to dict, keys present).
- Env precedence (`WASB_DEVICE`/`WASB_PYTHON`/`WASB_WEIGHTS`/`WASB_REPO_DIR`/`WASB_SCRATCH`) beats config; empty env var falls through (not clobbering) via `env(...) or w.get(...)`: VERIFIED.
- `_resolve_runner` alias set `native/native_wasb/wasb_native`, env-over-config precedence, lowercasing: VERIFIED. WASB + fusion(wasb) resolve to `NativeWasbRunner`; fusion(tracknet) and tracknet_hybrid raise clean `NotImplementedError`.

- Hydra-override parity for `device="cuda"` default: VERIFIED identical (`--device` default cuda == omitting it).
- `--set` override path through typed config: VERIFIED.
- Full suite (991 passed), ruff, mypy on changed files: all green.

One real bug (major), one minor doc inconsistency, one pre-existing nit.

49. user (2026-06-20T09:02:46.165Z)

Structured output provided successfully

50. assistant (2026-06-20T09:03:01.497Z)

I've completed the seam/config skeptic review of the `detector_impl='native'` wiring. Here is what I verified and found.

Verified working (no issues)

- **`model_dump` survival**: `WasbIndexConfig.device`/`python_bin` survive `config.get("indexing", {})` ? nested `wasb` dict ? `NativeWasbConfig.from_indexing_cfg` reads them correctly (confirmed empirically, not just by reading).
- **Env precedence**: `WASB_DEVICE`/`WASB_PYTHON`/`WASB_WEIGHTS`/`WASB_REPO_DIR`/`WASB_SCRATCH` beat config; an **empty** env var correctly falls through (the `env(...) or w.get(...)` idiom treats `""` as falsy) rather than clobbering config.
- **`_resolve_runner` seam**: alias set `native/native_wasb/wasb_native` all resolve; env (`RALLY_DETECTOR_IMPL`) beats config; case-insensitive. WASB and fusion(wasb-substrate) ? `NativeWasbRunner`; fusion(tracknet-substrate) and tracknet_hybrid ? clean `NotImplementedError` (the base refusal / fusion override fire as designed).
- **Hydra-override parity**: `device="cuda"` default keeps `wasb_infer.py`'s overrides byte-identical to the no-`--device` WSL invocation (`runner.device=cuda runner.gpus=[0]`), and the resumable manifest's `device` defaults to `"cuda"` for backward-compat.
- **`--set` override path**: `--set indexing.wasb.device=cpu` flows through the typed config into the native runner.
- **Suite/lint/types**: full `run_all_tests.py` = 991 passed; `ruff check` clean; `mypy` clean on all changed files.

Findings (returned via StructuredOutput)

1. **MAJOR** ? `backend/worker/__main__.py:183` `_resolve_train_detector` does not catch the `NotImplementedError` from `_resolve_runner`. With `detector_impl=native` + a non-WASB detector (`--segmenter tracknet_hybrid`, or `fusion_hybrid` with `fusion.substrate=tracknet`), the worker crashes with a raw traceback instead of the function's "print + return None ? `return 2`" pattern used for every other failure. Reproduced live. `cmd_infer` has the same exposure.
2. **MINOR** ? `native_wasb_runner.py` module docstring (lines 15-17) omits `WASB_DEVICE` and the `RALLY_SCRATCH`/`TMPDIR` scratch fallbacks that the code actually reads; disagrees with the `WasbIndexConfig` comment.
3. **NIT** ? `trajectory_hybrid.py:76` lowercases but doesn't `.strip()` the impl string, so `RALLY_DETECTOR_IMPL=' native '` silently falls through to the WSL runner. Pre-existing (the `stub` branch shared the gap), not introduced by this PR.

Key files: `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\worker\__main__.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\segmenters\trajectory_hybrid.py`, `C:\Users\avidu\Projects\badminton-highlight-indexer\backend\pipeline\detectors\native_wasb_runner.py`.